

Efficient frontier detection for robot exploration

The International Journal of
Robotics Research
2014, Vol. 33(2) 215–236
© The Author(s) 2013
Reprints and permissions:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/0278364913494911
ijr.sagepub.com



Matan Keidar and Gal A. Kaminka

Abstract

Frontier-based exploration is the most common approach to exploration, a fundamental problem in robotics. In frontier-based exploration, robots explore by repeatedly detecting (and moving towards) frontiers, the segments which separate the known regions from those unknown. A frontier detection sub-process examines map and/or sensor readings to identify frontiers for exploration. However, most frontier detection algorithms process the entire map data. This can be a time-consuming process, which affects the exploration decisions. In this work, we present several novel frontier detection algorithms that do not process the entire map data, and explore them in depth. We begin by investigating algorithms that represent two approaches: Wavefront Frontier Detector (WFD), a graph-search-based algorithm which examines only known areas, and Fast Frontier Detector (FFD), which examines only new laser readings data. We analytically examine the complexity of both algorithms, and discuss their correctness. We then improve by combining elements of both, to create two additional algorithms, called WFD-INC and WFD-IP. We empirically evaluate all algorithms, and show that they are all faster than a state-of-the-art frontier detector implementation (by several orders of magnitude). We additionally contrast them with each other and demonstrate the FFD and WFD-IP are faster than the others by one additional order of magnitude.

Keywords

Robotics, SLAM, exploration, laser, range, sensor, frontier

1. Introduction

The problem of exploring an unknown territory is a fundamental problem in robotics. The goal of exploration is to gain as much information as possible of the environment, through the robot's sensors, within bounded time. Applications of efficient exploration include search and rescue (Kitano et al., 1999), planetary exploration (Apostolopoulos et al., 2001) and military uses (Hougen et al., 2000). The most common approach to exploration is based on *frontiers*. A frontier is a segment that separates known (explored) regions from unknown regions. By moving towards frontiers, robots can focus their motion on discovery of new regions. Yamauchi (1997, 1998) was the first to show a frontier-based exploration strategy. His work led to many others (e.g. Burgard et al., 2000; Lau, 2003; Burgard et al., 2005; Sawhney et al., 2009).

Investigations of single- and multi-robot frontier-based exploration all share the key step of *frontier detection*, which is the process that identifies the current set of frontiers, given the current map (and/or history of the robots' perceptions). In almost all papers frontier detection is based on techniques borrowed from computer vision: edge detection and region extraction (see Section 2 for a detailed

discussion). The frontiers need to be detected again and again, as the exploration proceeds and areas that were once unknown become known.

To detect frontiers, existing methods process the entire map data with every execution to the algorithm. As a result, as maps grow large, state-of-the-art frontier detection algorithms can take a number of seconds to run, even on powerful computers. This significantly impacts the exploration strategy: If a large region is explored, the robot actually has to wait in its spot until the frontier detection algorithm terminates. Therefore, most exploration implementations call the frontier detection algorithm only when the robot arrives at its destination. Others do it only once some distance has been traveled, or once some time has passed since the last detection (see Section 2 for details).

This slows down the exploration process, and can cause inefficiencies in the exploration. We present two examples. First, we consider a common *single-robot case* (Figure 1),

Department of Computer Science, Bar-Ilan University, Ramat-Gan, Israel

Corresponding author:

Matan Keidar, Department of Computer Science, Bar-Ilan University, Building 216, Ramat-Gan 52900, Israel.
Email: matankdr@gmail.com

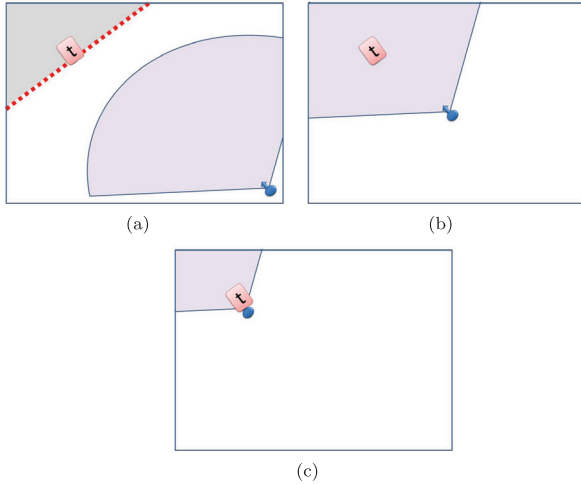


Fig. 1. A single-robot example. In (a) the robot is heading towards the marked target on the frontier. In (b) the target and all of the remainder are covered by the robot's sensors, but because the robot does not re-detect frontiers, it continues to move. In (c) the robot has reached the frontier, unnecessarily.

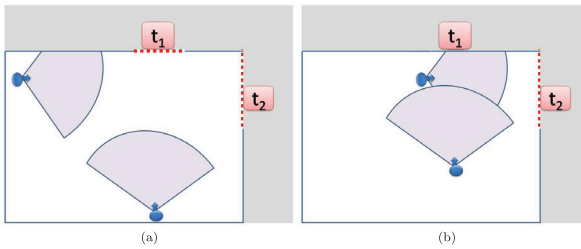


Fig. 2. A multi-robot example. In (a), the top robot (R_2) is heading towards the right target, t_2 ; the other robot (R_1) heads towards the top target t_1 . In (b) R_2 has reached its target, clearing both t_1 and t_2 , making R_1 's movements unnecessary.

where a robot exploring its environment detects a frontier and moves towards it (Figure 1(a)). Because of sensor coverage, the robot may in fact sense (and clear) all remaining unknown area (Figure 1(b)), but because it cannot call the frontier-detection mechanism, it continues to move unnecessarily (Figure 1(c)). Similarly, consider a *multi-robot case* (Figure 2). Here, two robots, R_1 and R_2 which are located on bottom and top, respectively, are exploring the environment, from their initial locations (Figure 2(a)). One of the robots passes by a target assigned to the other, and thus clearing it (Figure 2(b)). But because the other robot cannot continuously re-detect frontiers, it unnecessarily continues towards the covered target, instead of turning to more fruitful exploration targets.

In this work, we focus on significantly speeding up the frontier detection, common to all frontier-based exploration techniques. We introduce four algorithms for fast frontier detection.

The first, WFD (Wavefront Frontier Detector) is an iterative method that performs a graph search over already-visited map points (i.e. the known area), instead of the entire

map (which includes also unknown areas). It builds on ideas suggested in earlier work (Calisi et al., 2007) which were not evaluated as an alternative to the edge-detection state of the art (SOTA). However, as exploration progresses, the scanned area grows, and thus WFD cannot be expected to perform well in large areas.

Our second contribution is FFD (Fast Frontier Detector), a novel approach for frontier detection which processes raw sensor readings, and thus only scans areas that could contain frontiers. But because it works with raw sensor data, it requires extending the evolving map with additional data structures, so that frontiers are maintained even when they are no longer within sensor range. We describe these data structures in detail, focusing on fast implementations.

There is another way of looking at these algorithms. WFD and edge-detection methods use the occupancy grid maintained by a simultaneous localization and mapping (SLAM) mapper as a representation of the accumulated, processed sequence $\langle O_0, \dots, O_t \rangle$. They thus essentially ignore most of the sequence, and act (mostly) on the latest observation of the grid G_t , as it integrates all sensor readings up to time t . The edge-detection method processes G_t (the entire occupancy grid at time t). The WFD algorithm processes a subset, starting with the cell indicated by P_t (the latest robot position), within G_t . FFD uses R_t directly to detect current frontiers, and a modified occupancy grid data structure to maintain/delete past frontiers. We provide a theoretical complexity analysis for both algorithms, and discuss their analytical correctness.

Finally, we synthesized the different approaches of WFD and FFD into two novel algorithms: WFD-INC and WFD-IP. Both algorithms perform frontier detection on known regions but they differ from WFD by searching for frontiers only within the areas that were covered by the robot sensors since last call. They thus borrow from FFD the ability to maintain knowledge of frontiers not currently visible.

We provide a detailed evaluation of all algorithms, and contrast them with the SOTA. Specifically, we examine their performance in different types of environments, and on two different CPUs. We show that WFD and WFD-INC are faster than the SOTA by one to two orders of magnitude, and that FFD and WFD-IP are faster than WFD and WFD-INC by additional one to two orders of magnitude. The results make it possible to execute real-time frontier-detection on current-day robot CPUs, opening the way to novel frontier-based exploration strategies which were impractical until now.

This paper is organized as follows. Section 2 discusses related investigations. Sections 3 and 4 discuss the WFD and FFD algorithms, respectively, including their theoretical correctness. Section 5 discusses complexity of these methods. Section 6 shows how to combine elements of WFD and FFD to create two improved algorithms, WFD-INC and WFD-IP. Section 7 reports on empirical evaluation of all algorithms, contrasting with the SOTA. Finally, Section 8 concludes.

2. Related work

An outline of the exploration process can be described as follows: while there is an unknown territory, allocate to each robot a target point to explore and coordinate team members in order to minimize overlaps. In frontier-based exploration, targets are drawn from existing *frontiers*, segments that separate known and unknown regions (see Section 3.1 for definitions).

Most literature ignores the computational cost of frontier detection. Instead, there are two aspects that are often tackled in existing literature on exploration: deciding on next target to be explored and coordinating team members in order to minimize overlaps. The latter is not related to this work and so, we focus on the former, it may indirectly provide evidence as to the SOTA in frontier detection.

To the best of the authors' knowledge, all of the following works utilize a standard edge-detection method for computing the frontiers. They therefore recompute target locations whenever one robot has reached its target location or whenever a certain distance has been traveled by the robots or after a timeout event.

Yamauchi (1997, 1998) developed the first frontier-based exploration methods. The robots explore an unknown environment and exchange information with each other when they get new sensor readings. As a result, the robots build a common map (occupancy grid) in a distributed fashion. The map is continuously updated until no new regions are found. In his work, each robot heads to the *centroid*, the center of mass of the closest *frontier*. All robots navigate to their target independently while they share a common map. Frontier detection is performed only when the robot reaches its target.

Burgard et al. (2000, 2005) focus their investigation on a probabilistic approach for coordinating a team of robots. Their method considers the trade-off between the costs of reaching a target and the utility of reaching that target. Whenever a target point is assigned to a specific team member, the utility of the unexplored area visible from this target position is reduced for the other team members. In their work, frontier detection is carried out only when a new target is to be allocated to a robot.

Wurm et al. (2008) proposed to coordinate the team members by dividing the map into segments corresponding to environmental features. Afterwards, exploration targets are generated within those segments. The result is that in any given time, each robot explores its own segment.

In addition, Wurm (2011) claims that updating frontiers frequently is important in a multi-robot setting since the map will be updated not only by the robot assigned to a given frontier, but also by all of the robots in the team. In reasonably sized environments, Wurm suggests to call frontier detection on every time-step of the coordination algorithm. In real-world environments, he suggests executing the frontier detection algorithm according to the traveled distance (i.e. every 0.5–1 m) or on every second or whenever a new target is requested.

Stachniss (2006) introduced a method to make use of background knowledge about typical structures when distributing the team members over the environment. In his work, Stachniss computes new frontiers when there are new targets to be allocated. This happens whenever one robot has reached its designated target location or whenever the distance traveled by the robots or the elapsed time since last target assignment has exceeded a given threshold.

Berhault et al. (2003) proposed a combinatorial auction mechanism where the robots bid on a bunch of targets to navigate. The robots are able to use different bidding strategies. Each robot has to visit all of the targets that are included in his winning bid. After combining each robot's sensor readings, the auctioneer omits selected frontier cells as potential targets for the robots. Frontier detection is performed when creating and evaluating bids.

Visser and Slamet (2008) investigated how limited communication range affects multi-robot exploration. They proposed an algorithm which takes into account wireless constraints when selecting frontier targets. Visser (2011) suggests recomputing frontiers every 3–4 m, which, in his opinion, has a positive effect.

Juliá et al. (2012) surveyed the most important exploration methods according to different aspects such as team coordination and integration with the SLAM algorithm. None of the compared exploration strategies takes advantage of real-time frontier detection.

Lau (2003) presented a behavioral approach. The author assumes that all team members start from a known location. The team members follow the behavior and spread in the environment while updating a shared map. Frontier detection is called when the robot plans its next direction of movement.

Many other works omit details of their frontier-detection timing, and thus provide no evidence for the computational cost of frontier detection. However, they broadly refer to frontier detection as a central task as part of exploration, and thus signify its centrality within modern robot exploration systems.

Sawhney et al. (2009) presented an exploration method for both 2D and 3D environments. They showed a novel visibility per-time metric. Their method covers nearly the same number of points as other metric methods that can be found in literature. However, the time length of the paths is smaller. The outcome is reduced exploration time.

Simmons et al. (2000) proposed to assign a target destination to each robot in a way that maximizes the expected map knowledge over time. They proposed a bid-based heuristic. Each robot estimates its utility and cost until arriving various targets. According to this calculation, each robot creates bids. After receiving all bids, a central agent assigns a target to each robot considering minimization of the overlapping coverage of the team members.

Bouraqadi et al. (2009) proposed a flocking-based approach for solving the exploration problem, where robots

act according to the same set of rules. One of their rules (R5) makes the robot navigate towards the nearest frontier.

Ko et al. (2003) presented a decision-theoretic approach to the mapping and exploration problem. Their approach uses an adopted version of particle filters to estimate the position in the other robot's partial map.

Fox et al. (2010) presented an exploration and mapping distributed system for multi-robot exploring teams. Their system enables building a map of the environment and is robust against limited-range communication and against uncertainty of initial locations of the team members.

Zlot et al. (2002) coordinates a team of exploring robots by applying a market-based approach. The market architecture maximizes the exploration information gain while minimizing the travel distance in order to maximize the overall utility.

Kuipers and Byun (1991) presented a method for exploration and mapping for large-scale spatial environments. Their method utilizes the qualitative properties of large-scale environments before relatively error-prone geometrical attributes. This in contrast to traditional methods which utilize sensor data to build a geometrical representation of the environment and then extract topological structure from the geometric representation.

Rekleitis et al. (1997, 2001) presented a strategy of localization and exploration designed for small-scale and large-scale environments. Each pair of robots that are able to sense each other work together in order to reduce localization error. They modeled the free areas as a simple polygon with holes and use a well-known planar decomposition form. For large-scale areas, they use trapezoid decomposition and for small-scale areas, they apply a triangulation of the free space.

Stachniss et al. (2006) investigated the assignment of targets to the robot team members. They applied semantic information on the environment in order to better distribute the explorers over the explored environment.

Batalin and Sukhatme (2003) presented an exploration method which does not utilize a map for navigating in the environment. Their algorithm uses markers which are dropped off by the robot and aid the exploration.

Puig Valls (2010) presented an algorithm for coordinating the team members during the exploration task. Their method is based on K -means (KME) global optimization strategy.

Stoeter et al. (2000) presented a mechanism for controlling a robot team for missions of exploration and surveillance. Controlling the team members is performed through a hierarchical behavior tree. Their system is modular and enables adding new kinds of behaviors.

Konolige et al. (2003, 2006) presented *Centibots*, a framework which enables a large team of robots to be controlled, performing tasks of exploration, planning and collaboration in unknown environments.

Our work on WFD (Section 3) is independent from previous work, although ? mentions a frontier-detection algorithm that utilizes breadth-first search, similar to WFD.

However, ? does not provide details of the algorithm, nor evaluation of its performance, and so exact similarities and differences cannot be assessed.

3. Wavefront Frontier Detector

In this section we review the basic definitions and terms related to the domain of frontier-based exploration, as well as a formal definition of the frontier-based exploration problem (Section 3.1). We briefly discuss common SOTA techniques for performing a SLAM (Section 3.2), which underly existing approaches to exploration. We then present the WFD algorithm which searches for frontiers on the map (Section 3.3).

3.1. Frontier-based exploration: definitions

In this section we define and explain the terms that are used in the following sections. We assume that the robot in question uses an occupancy-grid map representation in the exploration process (Figure 3) within the map:

Unknown region is a territory that has not been covered yet by the robot's sensors.

Known region is a territory that has already been covered by the robot's sensors.

Open-space is a *known region* which does not contain an obstacle.

Occupied-space is a *known region* which contains an obstacle.

Occupancy-grid is a grid representation of the environment. Each cell holds a probability that represents if it is occupied.

Frontier is the segment that separates known (explored) regions from unknown regions. Formally, a frontier is a set of *unknown* points that each have at least one *open-space* neighbor.

Definition 1. Suppose that we are given a temporal sequence of observations $\langle O_0, \dots, O_t \rangle$ (time 0 to time t), where each observation O_x is a tuple $\langle G_x, P_x, R_x \rangle$ composed of: (i) the occupancy-grid G_x of time x ; (ii) the robot pose P_x (in occupancy-grid coordinates); and (iii) the range sensor readings R_x originating at the robot location (given in either ego-centric polar coordinates, or in occupancy-grid coordinates). The *frontier detection problem* is to return all frontiers existing at time t , given the sequence.

Existing algorithms for frontier detection rely on edge-detection methods. The algorithms systematically search for frontiers all over the occupancy-grid, i.e. both in known and unknown regions.

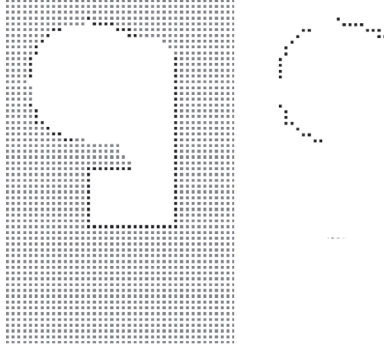


Fig. 3. Evidence grid, frontier points, extraction of different frontiers (from left to right).

3.2. SLAM methods

SLAM is a method utilized by autonomous mobile robots for building a map of an unknown environment while at the same time navigating the environment using the map. The term SLAM was originally developed by Leonard and Durrant-Whyte (1991) (see also Durrant-Whyte and Bailey, 2006). In SLAM both the trajectory of the mobile robot, its location and the location of the landmarks observed by the robot are estimated without using any *a priori* knowledge of the environment. SLAM is a concept and, hence, there are various algorithms in literature which deal with the SLAM problem.

Extended Kalman Filter The *Kalman filter* (KF) is a method for implementing Bayes filters. It was invented by Rudolph Emil Kalman (Welch and Bishop, 1995; Thrun et al., 2005) to apply filtering and perform prediction in linear systems. In a KF, beliefs are represented by moments (i.e. by the mean and the covariance). Markov assumptions are held and the posteriors are Gaussian. KFs are based on *linear* system dynamics. However, real-world systems are seldom linear in practice (e.g. robot which moves in a cyclic path). The *extended Kalman filter* (EKF) uses a non-linear function that describes the system dynamics. In EKF-SLAM systems, there is only one map (part of the system state) that is composed from the observed landmarks.

Particle filter A *particle filter* (also known as a *sequential Monte Carlo method*) is a model estimation method which is based on simulation. A particle filter is the online version of *Markov chain Monte Carlo* (MCMC) methods (Hastings, 1970; Geyer, 1992; Neal, 1993). With sufficient samples, a particle filter is often an alternative to EKF with the advantage that it approaches the Bayesian optimal estimate and so it is more accurate than EKF. Each particle is a possible hypothesis of what is the true world state at a certain time, that is, each particle is an instance of the system state at a certain time. By state we refer to a collection of variables such as the map of the explored environment, the robot position etc.

Algorithm 3.1 Wavefront Frontier Detector (WFD).

Require: $queue_m$ // queue, used for detecting frontier points from a given map
Require: $pose$ // current global position of the robot

```

1:  $queue_m \leftarrow \emptyset$ 
2: ENQUEUE( $queue_m, pose$ )
3: mark  $pose$  as “Map-Open-List”

4: while  $queue_m$  is not empty do
5:    $p \leftarrow$  DEQUEUE( $queue_m$ )
6:   if  $p$  is marked as “Map-Close-List” then
7:     continue
8:   end if
9:   if  $p$  is a frontier point then
10:     $NewFrontier \leftarrow$  EXTRACT-FRONTIER-2D( $p$ )
11:    save  $NewFrontier$ 
12:    mark all points of  $NewFrontier$  as “Map-Close-List”
13:   end if
14:   for all  $v \in N(p)$  do // get all neighbors of current frontier point
15:     if  $v$  not marked as one of {“Map-Open-List”, “Map-Close-List”} and  $v$  has at least one “Map-Open-Space” neighbor then
16:       ENQUEUE( $queue_m, v$ )
17:       mark  $v$  as “Map-Open-List”
18:     end if
19:   end for
20:   mark  $p$  as “Map-Close-List”
21: end while

```

In this work we focus on particle filter systems since in practice, they provide more accurate results.

3.3. WFD

We present a graph-search-based approach for frontier detection. The algorithm, WFD (Algorithms 3.1 and 3.2), processes the points on the map which have already been scanned by the robot sensors and, therefore, does not always process the entire map data in each run, but only the known regions.

WFD (Algorithm 3.1) is based on breadth-first search (BFS). First, the occupancy-grid point that represents the current robot position is enqueued into $queue_m$, a queue data structure used to determine the search order (Algorithm 3.1, lines 1–3).

Next, a BFS is performed (Algorithm 3.1, lines 4–21) in order to find all frontier points contained in the map. The algorithm keeps scanning only points that have not been scanned yet and represent open space (Algorithm 3.1, line 15). The above scanning policy ensures that only known regions (that have already been covered by the

Algorithm 3.2 Extract Frontier 2D.**Require:** p // frontier point**Ensure:** $NewFrontier$ // extracted frontier which was found starting from p

```

1: function EXTRACT-FRONTIER-2D( $p$ )
2:    $queue_f \leftarrow \emptyset$ 
3:    $NewFrontier \leftarrow \emptyset$ 
4:   ENQUEUE( $queue_f, p$ )
5:   mark  $p$  as “Frontier-Open-List”

6:   while  $queue_f$  if not empty do
7:      $q \leftarrow$  DEQUEUE( $queue_f$ )
8:     if  $q$  is marked as one of {“Map-Close-List”, “Frontier-Close-List”} then
9:       continue
10:    end if
11:    if  $q$  is a frontier point then
12:      add  $q$  to  $NewFrontier$ 
13:      for all  $w \in N(q)$  do // get all neighbors of
        current frontier point
14:        if  $w$  not marked as one of {“Frontier-Open-List”, “Frontier-Close-List”, “Map-Close-List”}
          then
15:          ENQUEUE( $queue_f, w$ )
16:          mark  $w$  as “Frontier-Open-List”
17:        end if
18:      end for
19:    end if
20:    mark  $q$  as “Frontier-Close-List”
21:  end while
22:  return  $NewFrontier$ 
23: end function

```

robot’s sensors) are actually scanned. The significance of this is that the algorithm does not have to scan the entire occupancy-grid each time.

Because frontier points are adjacent to open space points, all relevant frontier points will be found when the algorithm finishes (Algorithm 3.1, line 21). If a frontier point is found, a new BFS is performed in order to extract its frontier (Algorithm 3.2, lines 6–22). This BFS searches for frontier points only. Extracting the frontier is ensured because of the connectivity of frontier points.

At the end of the frontier extraction process (Algorithm 3.2, line 22), the extracted frontier data is saved to a set data structure that stores all frontiers found in the algorithm’s run.

In order to avoid rescanning the same map point and detecting the same frontier reachable from two frontier points, WFD marks map points with four indications:

1. Map-Open-List: points that have already been enqueued by the outermost BFS (Algorithm 3.1, line 16)
2. Map-Close-List: points that have already been dequeued by the outermost BFS (Algorithm 3.1, line 5)

3. Frontier-Open-List: points that have already been enqueued by the frontier extraction BFS (Algorithm 3.2, line 15)
4. Frontier-Close-List: points that have already been dequeued by the frontier extraction BFS (Algorithm 3.2, line 7)

The above marks indicate the status of each map point and determine whether there is a need to handle it in a given time.

The key innovation in WFD is that it prevents scanning unknown regions, since frontiers never appear there. However, it still searches all known space.

4. Fast Frontier Detector

We present a novel approach to the problem of frontier detection. The FFD algorithm (Section 4.1) avoids searching for frontiers both in known and unknown regions of the map; it only searches in the boundary between them. This significantly reduces the search area. However, the search-space reduction forces FFD to run in the background persistently. In order to be robust against map orientation changes caused by loop closures, FFD has to perform maintenance over previously detected frontiers (Section 4.1.4). At the end of this section, we prove that FFD is complete and sound (Section 4.2).

4.1. FFD

Unlike other frontier detection methods (including WFD), our proposed algorithm (Algorithm 4.1) only processes new laser readings which are received in real time. It therefore avoids searching both known and unknown regions. In doing this, we make use of the fact that by definition, frontiers represent the boundaries between the known and unknown regions of the environment (see Figure 3). Hence, scanning all unknown regions is definitely unnecessary. The FFD algorithm contains four steps (Algorithm 4.1), and can be called with every new scan.

4.1.1. Sorting The first step (Algorithm 4.2) sorts range readings based on their angle, i.e. based on the ego-centric polar coordinates with the robot as the origin. Normally, laser readings are given as a sorted set of polar coordinated points, making this sorting step unnecessary. However, if this is not the case, a sorting is needed to be applied on the received laser readings.

In this case, we assume that range readings are a set of Cartesian coordinated points, which consists of the locations of range hits ($\{(x_0, y_0), \dots, (x_n, y_n)\}$ where n is the number of readings). The naive method for converting Cartesian to polar coordinates uses two CPU time-consuming functions: *atan2* and *sqrt*.

Algorithm 4.1 Fast Frontier Detector (FFD): Outline.

Require: *frontiersDB* // data-structure that contains last known frontiers
Require: *pose* // current global position of the robot
Require: *lr* // laser readings which were received in current iteration. Each element is a 2D Cartesian point
Require: *activeArea* // data-structure that contains all points that lie inside the active area

```

1: // polar sort readings according to robot position
2: sorted ← SORT-POLAR(lr, pose)

3: // get the contour from laser readings
4: contour ← CONTOUR(sorted)

5: // extract new frontiers from contour
6: NewFrontiers ← EXTRACT-FRONTIERS-1D(contour)

7: // maintenance of previously detected frontiers
8: MAINTAIN-FRONTIERS(NewFrontiers, frontiersDB, activeArea)

```

Algorithm 4.2 Fast Frontier Detector (FFD): Sorting Stage.

Require: *lr* // set of laser readings
Require: *pose* // current robot position
Ensure: *sorted* // sorted laser readings according to polar coordinates

```

1: function POLAR-SORT(lr, pose)
2:   sorted ← ∅
3:   Ar ← ∅
   // set robot position as origin
4:   for all Point p ∈ lr do
5:     Ar ← polar ∪ {(p.x - pose.x, p.y - pose.y)}
6:   end for
7:   sorted ← SORT(Ar, POLARCOMPARATOR)
8:   return sorted
9: end function

10: function POLARCOMPARATOR(p1, p2)
11:   s ← (p1.x · p2.y - p2.x · p1.y) // calculate cross-product of input points
12:   if s > 0 then // check the sign of the cross-product
13:     return 1
14:   else if s < 0 then
15:     return -1
16:   else
17:     return 0
18:   end if
19: end function

```

To speed angle sorting, we use a cross-product (Cormen et al., 2001) to avoid converting Cartesian to polar coordinates, while still sorting the points based on polar angle. Given three Cartesian coordinated points

$$P_0 = (x_0, y_0), P_1 = (x_1, y_1), P_2 = (x_2, y_2)$$

Algorithm 4.3 Fast Frontier Detector (FFD): Contour Stage.

Require: *sorted* // sorted set of laser readings
Ensure: *contour* // the contour built from the sorted laser readings

```

1: function CONTOUR(sorted)
2:   prev ← POP(sorted)
3:   contour ← ∅
4:   for all Point curr ∈ sorted do
5:     line ← GET-LINE(prev, curr)
6:     for all Point p ∈ line do
7:       contour ← contour ∪ {p}
8:     end for
9:   end for
10:  return contour
11: end function

```

the *cross-product* is defined as

$$(p_1 - p_0) \times (p_2 - p_0) = (x_1 - x_0) \cdot (y_2 - y_0) - (x_2 - x_0) \cdot (y_1 - y_0)$$

If the result is positive, then $\overrightarrow{P_0P_1}$ is clockwise from $\overrightarrow{P_0P_2}$. Otherwise, it is counter-clockwise. If the result is zero, then the two vectors lie on the same line in the plane (i.e. the angle is the same).

Therefore, by examining the sign of the cross-product, we can determine the order of the Cartesian points according to polar coordinates, without calculating their actual polar coordinate value. This applies only five subtractions and two multiplications which are far less time-consuming than calling *atan2* and *sqrt*.

4.1.2. Contour In this step (Algorithm 4.3, lines 2–10) we use the angle-sorted laser readings. The output of the contour step is a contour which is built from the sorted laser readings. The algorithm computes the line that lies between each two adjacent points from the set. The line is computed by calling the function GET-LINE. In our implementation we use *Bresenham's line algorithm* (Bresenham, 2010). Next, all points that are represented by all of the lines (including the points from the laser readings set) are merged into a contour (Figure 4).

4.1.3. Detecting new frontiers In this step (Algorithm 4.4, lines 2–21) the algorithm extracts new frontiers from the previously calculated contour. There are three cases correspond to each two adjacent points in the contour:

1. **Current scanned point is not a frontier cell:** therefore, it does not contribute any new information about frontiers and can be ignored.
2. **Current and previous scanned points are frontier cells:** therefore, both points belong to the same frontier and current scanned point is added to last detected frontier.

Algorithm 4.4 Fast Frontier Detector (FFD): Extract Frontier Stage.

Require: *contour* // the contour built from the sorted laser readings

Ensure: *NewFrontiers* // list of new detected frontiers that lie on the contour

```

1: function EXTRACT-FRONTIERS-1D(contour)
2:   NewFrontiers  $\leftarrow \emptyset$  // list of new extracted frontiers
3:   prev  $\leftarrow \text{POP}(\text{contour})$ 
4:   if prev is a frontier cell then // special case
5:     create a new frontier in NewFrontiers
6:   end if
7:   for all Point curr  $\in$  contour do
8:     if curr is not a frontier cell then
9:       prev  $\leftarrow$  curr
10:    else if curr has been visited before then
11:      prev  $\leftarrow$  curr
12:    else if curr and prev are frontier cells then
13:      add curr to last created frontier
14:      prev  $\leftarrow$  curr
15:    else
16:      create a new frontier in NewFrontiers
17:      add curr to last created frontier
18:      prev  $\leftarrow$  curr
19:    end if
20:  end for
21:  return NewFrontiers
22: end function

```

3. **Current point is a frontier cell but the previous is not:** a new starting point of a frontier was detected. Hence, the algorithm creates a new frontier and adds the new starting point to it.

4.1.4. Maintaining previously detected frontiers FFD gains its speed by processing the laser readings only, rather than entire regions of the map. However, if the robot navigates towards a specific frontier, other previously detected frontiers are no longer updated because they are not covered by the robot's sensors. Thus, scanning the new received laser readings enables FFD to detect only *new* frontiers in each execution. In this step (Algorithm 4.5, lines 2–20), in order to get complete information about the frontiers, the algorithm performs maintenance over previously detected frontiers which are no longer covered in the range of the sensors. Only by joining together new detected frontiers and previously detected frontiers, we get the overall frontiers in current world state. This step has multiple goals: avoiding detection of new frontiers in an already scanned area (Section 4.1.4), eliminating frontier points which are no longer belong to frontiers (Section 4.1.4) and joining correctly the new detected frontiers together with previously detected frontiers (Section 4.1.4).

Algorithm 4.5 Fast Frontier Detector (FFD): Maintenance of Previously Detected Frontiers.

Require: *NewFrontiers* // list of new detected frontiers

Require: *frontiersDB* // data-structure that contains last known frontiers

Require: *activeArea* // data-structure that contains all points that lie inside the active area

```

1: function MAINTAIN-FRONTIERS(NewFrontiers,
   frontiersDB, activeArea)
2:   for all Point p  $\in$  ActiveArea do // eliminate previously
   detected frontiers
3:     if p is a frontier cell then
4:       // split the current frontier into two partial
       frontiers
5:       get the frontier f  $\in$  frontiersDB which
       enables p  $\in$  f
6:       f1  $\leftarrow$  f[1...p]
7:       f2  $\leftarrow$  f[(p + 1) ... |f|]
8:       remove f from frontiersDB
9:       add f1 and f2 to frontiersDB
10:    end if
11:  end for
12:  for all Frontier f  $\in$  NewFrontiers do // store new
   detected frontiers
13:    if f overlaps with an existing frontier
       ExistFrontier then
14:      merged  $\leftarrow$  f  $\cup$  ExistFrontier
15:      remove ExistFrontier from frontiersDB
16:      add merged to frontiersDB
17:    else
18:      create a new index and add f to frontiersDB
19:    end if
20:  end for
21: end function

```

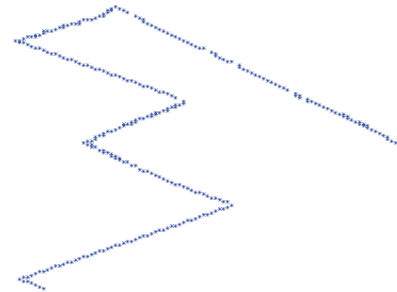


Fig. 4. Example of a produced contour.

The FFD data-structures In order to perform the maintenance step within a very short time as possible, FFD utilizes two data-structures which have a short access time. These data structures must maintain memory of frontiers between calls. Thus FFD has to have persistent memory, i.e. data structures that persist between calls. This is contrast to

other approaches that can be executed in a certain time, and only then.

Another thing to note is that in particle-filter-based systems (our focus in this work), each particle represents a possible hypothesis of the world state (including the robot position of course). The “best” particle is chosen according to a likelihood measurement. FFD requires the previously detected frontiers to be robust against map orientation changes caused by loop-closures of the mapping algorithm. Therefore, when a new laser reading is received, *each* particle executes its own instance of FFD algorithm on its own map, using its own data structures. More specifically, each particle performs maintenance with its own map because particles do not share maps. We describe the data structures for maintenance below.

The first, *grid of frontier indices*, is an extension of the occupancy grid (although it can be implemented as a separate entity). In addition to occupancy information, each grid cell contains a *frontier index*, pointing to a frontier to which the grid cell belongs, or NULL otherwise. The pointer points to a record stored in the frontier database (described below). In our implementation, we used integer index values. After accessing a grid cell (Algorithm 4.5, line 3), querying for its frontier index is $O(1)$.

The second, *frontier database*, maps frontier indices (pointers) to sets of points (*frontierDB* requirement of Algorithms 4.1 and 4.5). All detected frontiers are stored in this data-structure. We use it to map frontier index to the actual set that contains the points in world coordinates. In our implementation, we use the default C++ implementation of a map template, which is implemented as a self-balancing binary search tree. Therefore, assuming that n represents the number of frontiers stored in the database, searching for a frontier index takes $O(\log n)$, inserting a new frontier takes $O(\log n)$ and removing a frontier index takes $O(\log n)$, although a (hash) table lookup implementation can make this faster.

Avoiding re-detection of same frontier FFD detects new frontiers by processing laser readings only. Hence, FFD might detect the same frontier again and classify it as a new frontier if the robot did not change its position during two following FFD executions. Moreover, if the robot travels back to an already visited region, no new frontiers should be detected. FFD has to distinguish between laser readings from between time frames. Otherwise, FFD might wrongly detect a new frontier which lies within an already scanned area.

Therefore, we keep track of the number of sensor visits (sensor covers) of each map cell. The definition of a frontier point is now expanded: a frontier point is a point which represent an unknown region, has at least one open-space neighbor and has not been scanned before. Given a contour, the detection of new frontiers ignores points that have already been scanned by the laser sensors and treats them as non-frontier points (Algorithm 4.4, lines 10–11).

Figure 5 demonstrates the necessity of the above. Figure 5(a) shows frontiers extraction without tracking the number of visits. It can be seen that there are frontiers that lie inside an open space area. This is absolutely wrong because frontiers are supposed to be positioned on the boundaries between known and unknown regions. In contrast, Figure 5(b) shows frontiers extraction with avoidance of re-detecting same frontiers. It can be seen that every frontier separates known and unknown regions.

Eliminating previously detected frontiers In order to complete the process, points which are no longer in frontiers (i.e. were covered by the robot’s sensors) have to be eliminated. Lines 2–11 in Algorithm 4.5 contain the elimination logic applied by FFD.

Let t_i be a time frame and lr_{t_i} be the laser readings which were received in time frame t_i . In order to perform maintenance in a specific time, we define the *active area* of time frame t_i to be the blocking rectangle that can be constructed using the farthest laser readings of lr_{t_i} , relative to the robot position in time frame t_i .

$$x_{min} = \min(\{x \mid x \in lr_{t_i}\}), y_{min} = \min(\{y \mid y \in lr_{t_i}\})$$

$$x_{max} = \max(\{x \mid x \in lr_{t_i}\}), y_{max} = \max(\{y \mid y \in lr_{t_i}\})$$

$$ActiveArea_{t_i} = \{(x, y) \mid x_{min} \leq x \leq x_{max}, y_{min} \leq y \leq y_{max}\}$$

The active area’s rectangle is constructed from the following vertices: (x_{min}, y_{min}) , (x_{min}, y_{max}) , (x_{max}, y_{max}) , (x_{max}, y_{min}) . The rectangle is an approximation to the real active area that is actually bounded within the laser readings.

By processing received laser readings, FFD extracts new frontiers. However, in order to get the complete world’s frontiers state, points that are no longer on frontiers have to be eliminated. FFD maintains a frontier database which maps an integer (frontier index) to a set of points (frontier).

An unknown region is classified as a known region only if it is covered by the robot’s sensors. FFD gets its input from the new received laser readings, and thus only regions that are covered by the robot’s sensors might contain frontiers that have to be eliminated. Thus, if there are frontiers that need to be eliminated, they must lie inside the *active area*. Hence, the active area is a key feature in the process of maintaining frontiers. FFD scans each point that lies inside the active area and checks if it was previously belonged to a frontier. The check can be performed very fast as explained before. If the current scanned point was belonged to a frontier, the current scanned point is removed from the frontier and the frontier is split into two partial frontiers using the current scanned point as a pivot (Algorithm 4.5, lines 6–7).

In the end of this process, all no-longer frontier points in the frontier database are removed and the database contains only points that are still valid frontiers.

Storing new detected frontiers In the last phase of the maintenance step (Algorithm 4.5, lines 12–20) new

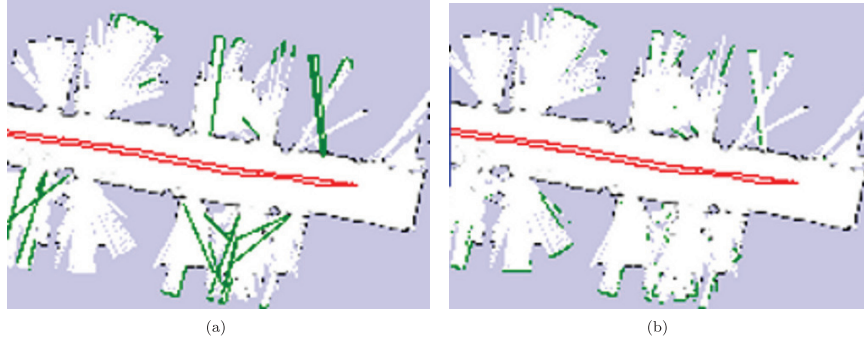


Fig. 5. An example of re-detecting same frontiers: (a) incorrectly re-detected frontiers; (b) correct detection.

detected frontiers are stored in the frontier database alongside with existing valid frontiers. For each new detected frontier, FFD checks whether it overlaps with an already existing frontier. This comparison can be performed in a short time using the matrix of frontier indices. Each frontier point is queried in $O(1)$ operations. If an overlap is found, the frontier is merged with the frontier that it is overlapped with. If no overlap is found, then the frontier is inserted into the frontier database.

4.2. FFD is sound and complete

We show that Algorithm 4.1 is sound and complete. We begin with a lemma that demonstrates that FFD always recognizes new frontiers (i.e. frontiers that appeared at time t , but did not exist before). This will then be used to prove the completeness of FFD.

Lemma 4.1. *Suppose that f is a frontier point at time t , which was not a frontier point at any time s , where $s < t$. Then FFD will mark f as a frontier given observation O_t .*

Proof. Let f be a valid frontier point in time t and was not classified as frontier in time $s < t$. Since f is a valid frontier point, then it has a value of *Unknown* and has at least one *Open Space* neighbor at time t . Assume towards a contradiction that FFD did not recognize f as a frontier point. First, let us show that f is contained in the contour handled in Algorithm 4.4, lines 2–21. Since f is a valid frontier point, then it has a value of *Unknown* and has at least one *Open Space* neighbor in time t . The point f cannot be located wholly within an *Unknown* region because it must have at least one *Open Space* neighbor. Also, the point f cannot be located wholly within a *known* region since f is a valid frontier point and hence, its value is *Unknown*. Therefore, f must be located on the contour itself. Lines 2–21 in Algorithm 4.4 handle points on the contour, which we have just shown f is on. In these lines, the FFD algorithm scans all contour points sequentially and specifically searches for frontier points. Because it scans all points on the contour, and we have shown that f is on the contour, it follows that f would be detected, contradicting the assumption that FFD did not recognize f as a frontier point at time t . $\square$

We now turn to proving the completeness of the FFD algorithm.

Theorem 4.2. *Let f be a valid frontier point at time t . Then FFD will mark f as a frontier point given the sequence of observations $\langle O_0, \dots, O_t \rangle$.*

Proof. Two cases should be examined.

Case 1: f is a new frontier point at time t . Trivially, this case is handled directly by Lemma 4.1.

Case 2: f was a new frontier point at time s , where $s < t$. Let s be the earliest time in which f was a frontier. Based on Lemma 4.1, it follows that it was detected at this time. All that remains to show is that given that f is still valid at time t , FFD will maintain knowledge of it from time s and report on it. If f is still a valid frontier point at time t , then it has not been covered yet by the robot's sensors. Otherwise, it would no longer contain an *Unknown* value and, hence, would not be a valid frontier point. So if it was not yet covered, it must be a frontier point that is maintained by FFD. The only way in which f can be eliminated from being classified as a frontier point is done by Algorithm 4.5, lines 2–11. In these lines, FFD scans all points that are covered by the robot's sensors and checks if any points should be eliminated (Algorithm 4.5, line 3). Since f is not covered by the sensors, then it will not be scanned and eliminated in time $t \Rightarrow f$ remains classified as a frontier by FFD.

In both cases we show FFD will recognize f to be a valid frontier at time t . Since Theorem 4.2 is true for any frontier point valid at time t , it follows that FFD is complete. $\square$

To show the soundness of FFD, we must demonstrate that there does not exist a case where FFD marks a point $\hat{f}$ as a frontier when it is not.

Theorem 4.3. *Let $\hat{f}$ be an arbitrary point in the occupancy grid, which is not a frontier at time t . Then FFD will not return $\hat{f}$ as a frontier point, given the sequence of observations $\langle O_0, \dots, O_t \rangle$.*

Proof. Assuming that $\hat{f}$ is an arbitrary point which is not a frontier point at time t , then $\hat{f}$ is either contains value different from *Unknown* or all of its adjacent values are different from *Open Space*. We will examine two cases.

Case 1: $\hat{f}$ is marked as a new frontier. Suppose, towards a contradiction, that FFD detects $\hat{f}$ as a new frontier (i.e. true at time t , but not a frontier in time s , where $s < t$). Since detection of new frontier points (Algorithm 4.4, lines 2–21) considers only points on the contour, it follows that $\hat{f}$ must be located on the contour *and* detected by Algorithm 4.4, lines 2–21. However, Algorithm 4.4, line 8, specifically avoids classifying non-frontier points as frontiers. Since $\hat{f}$ is a non-frontier point, it is ignored by FFD. Therefore, $\hat{f}$ cannot be marked as a new frontier $\Rightarrow$ contradicting the assumption that it is detected by FFD as a new frontier. Case 1 is impossible.

Case 2: $\hat{f}$ is an old frontier but was not eliminated by the maintenance routine. Suppose, towards a contradiction, that $\hat{f}$ is located inside the active area and is not eliminated by the maintenance section. Therefore, $\hat{f}$ is a point that was covered by the robot's sensors and no longer contains an *Unknown* value, yet is still marked as a frontier by the FFD algorithm. We remind the reader that in order to maintain frontier points across runs, each point in the grid keeps a value which contains NULL if the point is not a frontier point or the index of the frontier to whom it belongs. Therefore, in Algorithm 4.5, line 3, FFD scans all points in the active area and checks if they contain a frontier index. When FFD scans $\hat{f}$, it finds out that it contains a valid frontier index (because it has previously belonged to a valid frontier) and continues executing Algorithm 4.5, lines 5–9. In these lines, FFD checks and removes from the frontier database all points that are no longer frontier points and previously were frontier points. Thus, $\hat{f}$ will be eliminated after scanning the active area, contradicting the assumption that $\hat{f}$ was not eliminated.

Since in both cases we show that FFD necessarily eliminated $\hat{f}$ from the valid frontier list, it follows that if $\hat{f}$ is not a frontier-point at time t , it would not be marked as such by FFD. Since Theorem 4.3 holds for any arbitrary point, it follows that FFD never incorrectly marks a non-frontier point as a frontier. It is thus sound. $\square$

5. Complexity analysis

In this section, we show a theoretical complexity analysis of the WFD and FFD algorithms. In Section 5.1, we discuss WFD and in Section 5.2, we discuss FFD.

5.1. WFD complexity analysis

As shown in Section 3, WFD is based on BFS over the occupancy grid. Within every call to WFD, it scans all open-space regions for frontier points. When a frontier point is found, WFD performs another BFS in order to extract its frontier. Regarding a graph $G = (V, E)$, the upper bound of BFS run-time is $\mathcal{O}(E + V)$, full proof can be found in Cormen et al. (2001). Therefore, the complexity of scanning the open-space regions is linear in size of the area of the open-space regions. Moreover, frontier points are always located

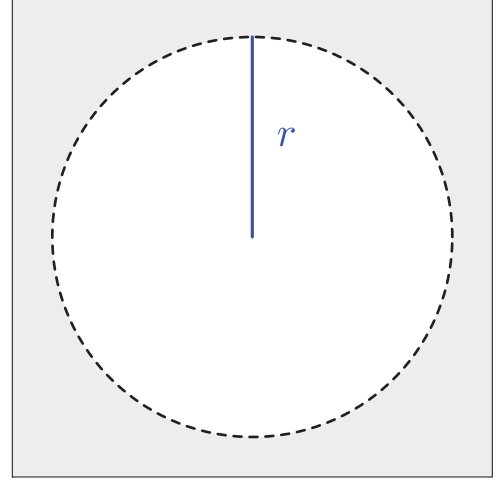


Fig. 6. WFD best case: perimeter of *open-space* regions is small as possible.

on the edges of the open-space regions. Thus, extracting frontiers from given frontier points is also linear in size of the perimeter of the open-space region. Hence, WFD's run-time complexity is linear in size of the area (denoted $\mathcal{S}(\cdot)$) and perimeter ($\mathcal{P}(\cdot)$) of the open-space region and can be formulated as

$$\mathcal{O}\left(\underbrace{\mathcal{S}(\text{open} - \text{space})}_{\text{area}} + \underbrace{\mathcal{P}(\text{open} - \text{space})}_{\text{perimeter}}\right) \quad (1)$$

In the following sections we discuss WFD's best case and worst case. The reader should be aware to the fact that for a given map and a time stamp, the open-space area is determined by the trajectory of the robot. However, its perimeter length affects the run-time and two cases should be examined:

5.1.1. WFD best case The perimeter of the open-space regions is minimal relatively to the area of the open-space regions. This can be shown in Figure 6.

In this case we denote the area of the shape as S_{open} and its perimeter as P_{opt} . Their values are

$$S_{\text{open}} = 4\pi r^2 \quad (2)$$

$$P_{\text{opt}} = 2\pi r \quad (3)$$

5.1.2. WFD worst case In the worst case we would like to maximize the length of the perimeter while keeping the total area of the open-space regions. Therefore we use a polygon as an approximation to a circle. We take an inner regular polygon and build triangles on each side. The level of accuracy is determined by k , the number of vertices. For a given k value we denote the total area of the shape by S_k and the total perimeter length by P_k . In order to bound an area, at least three vertices have to be used. As mentioned previously, all areas are equal and thus, for each $k \in \mathbb{N} \setminus \{0, 1, 2\}$

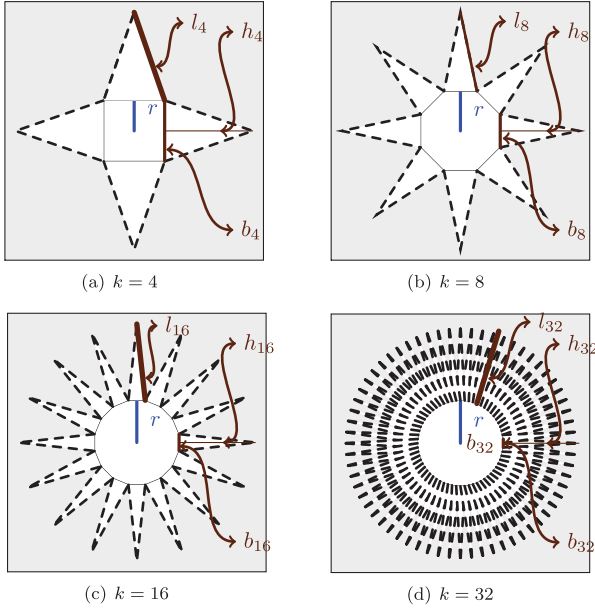


Fig. 7. WFD worst case: different values of k .

holds: $S_k = S_{open}$. Figure 7 demonstrates the results of different k values.

5.1.3. The general case Assume that we have a general shape with an arbitrary number of vertices, k , and a radius, r . Let b_k be the base of the inner polygon that is located inside the open-space regions (examples can be found in Figure 7). The length of base b_k is given by the formula

$$b_k = 2r \cdot \tan \frac{\pi}{k} \quad (4)$$

Let h_k be the height of an outer triangle. The length of h_k is given by

$$\begin{aligned} S_k &= S_{open} \\ k \cdot \frac{r \cdot b_k}{2} + k \cdot \frac{h_k \cdot b_k}{2} &= 4\pi r^2 \\ k \cdot r \cdot b_k + k \cdot h_k \cdot b_k &= 8\pi r^2 \\ h_k &= \frac{8\pi r^2 - k \cdot r \cdot b_k}{k \cdot b_k} = \frac{8\pi r^2}{k \cdot b_k} - r \end{aligned} \quad (5)$$

Let l_k be the length of an outer triangle side. The length of l_k is given by

$$l_k = \sqrt{(h_k)^2 + \left(\frac{b_k}{2}\right)^2} \quad (6)$$

Therefore, the length of the polygon's perimeter equals to the total length of the outer edges of the triangles. We can calculate the perimeter of the polygon using

$$P_k = 2k \cdot l_k \quad (7)$$

We now express the perimeter P_k as a function of the area S_{open} :

$$\begin{aligned} P_k &= 2k \cdot l_k \\ &= 2k \sqrt{(h_k)^2 + \left(\frac{b_k}{2}\right)^2} \\ &= 2k \sqrt{\left(\frac{8\pi r^2}{k \cdot b_k} - r\right)^2 + \left(\frac{2r \cdot \tan \frac{\pi}{k}}{2}\right)^2} \\ &= 2k \sqrt{\left(\frac{8\pi r^2}{k \cdot b_k} - r\right)^2 + \left(r \cdot \tan \frac{\pi}{k}\right)^2} \end{aligned} \quad (8)$$

Equations (2) and (4) can be combined to give

$$\begin{aligned} P_k &= 2k \sqrt{\left(\frac{2S_{open}}{k \cdot 2r \cdot \tan \frac{\pi}{k}} - r\right)^2 + \left(r \cdot \tan \frac{\pi}{k}\right)^2} \\ &= 2k \sqrt{\left(\frac{S_{open}}{k \cdot r \cdot \tan \frac{\pi}{k}} - r\right)^2 + \left(r \cdot \tan \frac{\pi}{k}\right)^2} \\ &= 2k \sqrt{\left(\frac{S_{open}}{k \cdot r \cdot \tan \frac{\pi}{k}}\right)^2 - 2 \frac{r \cdot S_{open}}{k \cdot r \cdot \tan \frac{\pi}{k}} + r^2 + r^2 \cdot \tan^2 \frac{\pi}{k}} \\ &= 2k \sqrt{\left(\frac{S_{open}}{k \cdot r \cdot \tan \frac{\pi}{k}}\right)^2 - 2 \frac{S_{open}}{k \cdot \tan \frac{\pi}{k}} + r^2 \left(1 + \tan^2 \frac{\pi}{k}\right)} \\ &= 2k \sqrt{\left(\frac{S_{open}}{k \cdot r \cdot \tan \frac{\pi}{k}}\right)^2 - 2 \frac{S_{open}}{k \cdot \tan \frac{\pi}{k}} + \frac{r^2}{\cos^2 \frac{\pi}{k}}} \\ &= \sqrt{\left(\frac{2k \cdot S_{open}}{k \cdot r \cdot \tan \frac{\pi}{k}}\right)^2 - 2 \frac{4k^2 \cdot S_{open}}{k \cdot \tan \frac{\pi}{k}} + \frac{4k^2 \cdot r^2}{\cos^2 \frac{\pi}{k}}} \\ &= \sqrt{\left(\frac{2 \cdot S_{open}}{r \cdot \tan \frac{\pi}{k}}\right)^2 - 8 \frac{k \cdot S_{open}}{\tan \frac{\pi}{k}} + 4 \frac{k^2 \cdot r^2}{\cos^2 \frac{\pi}{k}}} \end{aligned} \quad (9)$$

The only case that yields a division by zero in Equation (9) happens when $k = 2$. That causes both $\tan \frac{\pi}{k}$ and $\cos \frac{\pi}{k}$ to return the value of ∞ . In any other values of k , the functions are well-defined. Since we work on a discrete domain, the number of vertices the polygon that approximate the open-space area is bounded by the number of open-space points, which in our case equals to S_{open} :

$$k \leq S_{open} \quad (10)$$

In the range $[3, \infty)$ the function $f(x) = \frac{1}{\cos x}$ is a monotonic decreasing function and therefore gets a maximal value of 2. This yields

$$\frac{1}{\cos \frac{\pi}{k}} \leq 2 \quad \text{for } k \in \mathbb{N} \setminus \{0, 1, 2\} \quad (11)$$

Therefore, Equations (9), (10) and (11) can be combined and we obtain

$$P_k \leq \sqrt{\left(\frac{2 \cdot S_{open}}{r \cdot \tan \frac{\pi}{S_{open}}}\right)^2 - 8 \frac{S_{open}^2}{\tan \frac{\pi}{S_{open}}} + 2 \cdot S_{open}^2 \cdot r^2} \quad (12)$$

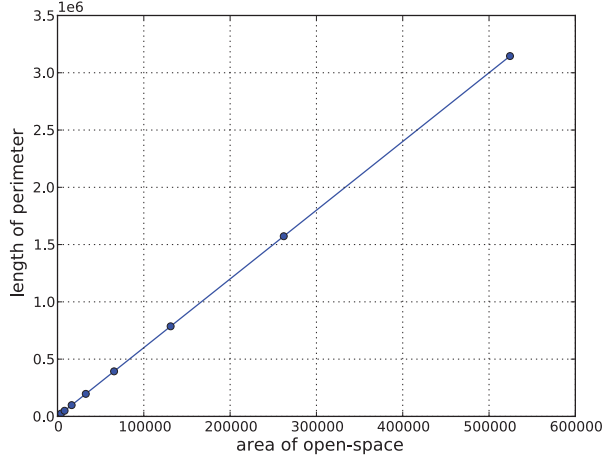


Fig. 8. The linearity relation between the area of open-space region and its perimeter (Equation (9)). The x-axis contains samples of different open-space regions in sizes of $2^2, 2^3, \dots, 2^{20}$ and the y-axis contains their perimeter sizes.

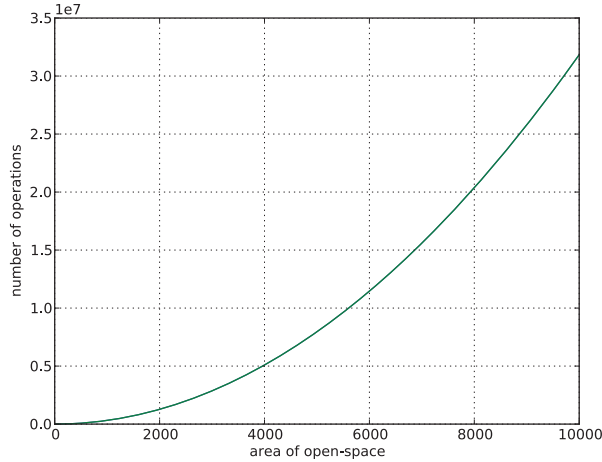


Fig. 9. WFD's complexity function, Equation (13).

Therefore, when we join Equation (12) together with (1), we get the overall run-time complexity of WFD algorithm in terms of the open-space area, S_{open} :

$$\mathcal{O}\left(S_{open} + \sqrt{\left(\frac{2 \cdot S_{open}}{r \cdot \tan \frac{\pi}{S_{open}}}\right)^2 - 8 \frac{S_{open}^2}{\tan \frac{\pi}{S_{open}}} + 2 \cdot S_{open}^2 \cdot r^2}\right)$$

After omitting constants,

$$\mathcal{O}\left(S_{open} + \sqrt{\left(\frac{S_{open}}{r \cdot \tan \frac{\pi}{S_{open}}}\right)^2 - \frac{S_{open}^2}{\tan \frac{\pi}{S_{open}}} + S_{open}^2 \cdot r^2}\right) \quad (13)$$

Figure 8 shows an evaluation of the open-space region's perimeter length as a function of the total area of open-space regions.

Figure 9 shows the graph for WFD final complexity function (Equation (13)), in the range of $S_{open} \in [0, 10000]$.

5.2. FFD complexity analysis

In Section 5.1, we showed that the run-time complexity of WFD is bounded by Equation (1). Allegedly, it may seem that FFD's complexity is contained within WFD since FFD searches only inside the active area where all of its points are open-space. However, this is not true since FFD has to persistently run in the background. It may thus run a number of times by every single run of WFD. Moreover, it is run for each particle in particle-based mapping systems. As shown in Section 4, FFD contains four stages. We now analyse the complexity of each stage separately.

5.2.1. Sorting stage (Algorithm 4.2) FFD performs polar-sorting of laser readings. FFD utilizes cross-products instead of actually calculating angle and radius (which are relatively very time-consuming calculations). As explained in Section 4, in order to determine the order of the points received from the laser readings, we calculate the cross-product between them. Cross-product calculation is performed in a constant time and sorting is performed in time of $\mathcal{O}(n \log n)$. We denote the number of laser readings received in each measurement as l_r and, therefore, the total complexity of this stage is

$$\mathcal{O}(l_r \log l_r) \cdot \mathcal{O}(1) = \mathcal{O}(l_r \log l_r) \quad (14)$$

5.2.2. Contour stage (Algorithm 4.3) FFD scans each two adjacent points received from the polar-sorted laser readings. For each two adjacent points FFD calls *Bresenham's line algorithm* which returns a set of the points that lie between them. *Bresenham's line algorithm* is performed in linear time complexity. In the end, FFD merges all received lines into a single contour. We denote the Euclidean distance between points p_i and p_j as d_{p_i, p_j} . Therefore, the total complexity of this stage is the total distance between each two adjacent points, which is denoted as $c^{(t)}$, the length of the contour in time t :

$$\mathcal{O}\left(\sum_{p_i \in l_r} d_{p_{i-1}, p_i}\right) = \mathcal{O}(c^{(t)}) \quad (15)$$

5.2.3. Detecting new frontiers stage (Algorithm 4.4) FFD scans the contour from the previous stage and extracts new frontiers (if available of course). Each newly detected frontier found is added to a list (which will be used in next stage) and, hence, the addition is performed in a constant time. Lines 2–6 handle the special case when the first point on the contour is a frontier point. All actions are performed in a constant time. Therefore, total complexity of this stage is proportional to the length of the given contour:

$$\underbrace{\mathcal{O}(1)}_{\text{special case}} + \underbrace{\mathcal{O}(c^{(t)}) \cdot \mathcal{O}(1)}_{\text{general case}} = \mathcal{O}(c^{(t)}) \quad (16)$$

$$\begin{aligned}
& \mathcal{O} \left(\underbrace{l_r \log l_r}_{\text{sorting}} + \underbrace{c^{(t)}}_{\text{contour}} + \underbrace{c^{(t)}}_{\text{detection}} + \overbrace{A^{(t)} \cdot (f_{\max}^{(t)} + \log n_f^{(t)}) + n_{\text{new}}^{(t)} \cdot (f_{\max}^{(t)} + \log n_f^{(t)})}^{\text{frontiers maintenance}} \right) \\
& = \mathcal{O} \left(l_r \log l_r + c^{(t)} + (A^{(t)} + n_{\text{new}}^{(t)}) \cdot (f_{\max}^{(t)} + \log n_f^{(t)}) \right) \quad (19)
\end{aligned}$$

eliminate existing frontiers
add new frontiers

5.2.4. Maintenance stage (Algorithm 4.5)

Elimination of previous frontiers (lines 2–11) FFD scans each point which lies inside the active area and checks if it was previously belonged to a frontier. Checking a specific point is performed in a constant time since all points in map already store a frontier index. The index is used as a lookup index in the frontier database (full implementation details can be found in Section 4). We denote the bounding rectangle (the active area) in time t as $A^{(t)}$. Hence, scanning all points in the active area can be done in time of $\mathcal{O}(A^{(t)}) \cdot \mathcal{O}(1) = \mathcal{O}(A^{(t)})$. We denote the length of the longest frontier that exists in the frontier database in time t as $f_{\max}^{(t)}$ and the number of frontiers that are stored in the frontier database in time t as $n_f^{(t)}$. If a previous frontier point is found, then FFD searches for its frontier in the database and removes this point from the frontier. Therefore, for a specific frontier point it takes $\mathcal{O}(\log n_f^{(t)})$ to find the frontier in the frontier database by the lookup index (we used a self-balancing binary search tree in our implementation), $\mathcal{O}(f_{\max}^{(t)})$ to locate the specific point in the found frontier and $\mathcal{O}(1)$ to remove it from the found frontier. Therefore, total complexity for this stage is

$$\mathcal{O}(A^{(t)}) \cdot \mathcal{O}(f_{\max}^{(t)} + \log n_f^{(t)}) = \mathcal{O}(A^{(t)} \cdot (f_{\max}^{(t)} + \log n_f^{(t)})) \quad (17)$$

Adding new frontiers (lines 12–20) FFD scans each frontier point within all new detected frontiers that were found at the detection stage. We denote the number of new frontiers that were found in time t as $n_{\text{new}}^{(t)}$ and, hence, scanning all new detected frontiers that are found in time t is performed in complexity of $\mathcal{O}(n_{\text{new}}^{(t)})$. Next, FFD checks if each new detected frontier point belongs to another previously detected frontier. As mentioned before, this check can be performed in a constant time since each point stores a frontier lookup index. Therefore, scanning all points in a specific frontier is performed in time of $\mathcal{O}(f_{\max}^{(t)})$ and searching the frontier whose index is stored within the point is performed in time of $\mathcal{O}(\log n_f^{(t)})$. In the end, the new frontier is merged with the existing frontier which is performed in a constant time. The total complexity of this stage is

$$\mathcal{O}(n_{\text{new}}^{(t)}) \cdot \mathcal{O}(f_{\max}^{(t)} + \log n_f^{(t)}) = \mathcal{O}(n_{\text{new}}^{(t)} \cdot (f_{\max}^{(t)} + \log n_f^{(t)})) \quad (18)$$

5.2.5. Combining all stages By joining together Equations (14), (15), (16), (17) and (18), we get the total complexity for one iteration:

All new detected frontiers are found only inside the active area. Thus, the number of new detected frontiers is bounded by size of the active area:

$$n_{\text{new}}^{(t)} \leq A^{(t)} \quad (20)$$

All new detected frontiers lie within the contour. In the worst case, all points that lie within the contour belong to the longest frontier. Hence, we can bound the length of the longest frontier by the contour's length:

$$f_{\max}^{(t)} \leq c^{(t)} \quad (21)$$

Therefore, by joining the upper bounds from Equations (20) and (21) together with (19) we bound the complexity of FFD by

$$\begin{aligned}
& \mathcal{O}(l_r \log l_r + c^{(t)} + (A^{(t)} + A^{(t)}) \cdot (c^{(t)} + \log n_f^{(t)})) \\
& = \mathcal{O}(l_r \log l_r + c^{(t)} + A^{(t)} \cdot (c^{(t)} + \log n_f^{(t)})) \quad (22)
\end{aligned}$$

The size of the active area can be bounded by the maximum range of the laser. The largest active area rectangle is received only when the laser scans an open area. In this case, the active area is a rectangle that bounds a circle whose radius is equal to the maximum laser range:

$$A^{(t)} \leq (2l_m) \times (2l_m) = 4(l_m)^2 \quad (23)$$

Joining together (22) and (23), we have

$$\begin{aligned}
& \mathcal{O}(l_r \log l_r + c^{(t)} + 4(l_m)^2 \cdot (c^{(t)} + \log n_f^{(t)})) \\
& = \mathcal{O}(l_r \log l_r + c^{(t)} + 4(l_m)^2 \cdot c^{(t)} + 4(l_m)^2 \cdot \log n_f^{(t)}) \\
& = \mathcal{O}(l_r \log l_r + (1 + 4(l_m)^2) \cdot c^{(t)} + 4(l_m)^2 \cdot \log n_f^{(t)}) \quad (24)
\end{aligned}$$

The maximal laser range and the number of laser readings (l_m and l_r , respectively) do not change during the execution. Hence, they both can be considered as constants and therefore, can be removed from the asymptotic upper bound:

$$\mathcal{O}(c^{(t)} + \log n_f^{(t)}) \quad (25)$$

5.3. Summary

In this section, we showed running-time upper-bounds for both WFD and FFD algorithms (Sections 5.1 and 5.2, respectively). Until now, we discussed FFD's run-time complexity in a single execution. However, as stated in Section 4, FFD has to persistently run in the background. More precisely, in order to maintain previously detected frontiers, FFD is executed when a new laser reading is received. Traditional frontier detectors are executed on demand, usually when a map-event has occurred. A map-event is an event which is raised by the SLAM implementation when a sufficient amount of new data was received in order to produce a new map. In order to compare FFD to other traditional frontier detectors, we define l_ω as the frequency of the laser sensor (e.g. how many laser readings are received in a time unit of 1 second). In addition, we denote the worst-case elapsed time between two following map-events in the (i.e. in times t_i and t_{i+1}) by t_m . Finally, we denote P_n as the number of particles in the SLAM implementation. In implementations containing only one map, such as EKF-based systems, the value of P_n is 1. The reason for this is that, as stated in Section 4, each particle has to maintain its own map and thus, run its own instance of FFD. Thus, Equation (25) now considers the frequency of the laser sensor and we get

$$\mathcal{O}\left(P_n \cdot (t_m \cdot l_\omega) \cdot \left(c^{(t)} + \log n_f^{(t)}\right)\right) \quad (26)$$

It might seem that in Equation (26) the value of $P_n \cdot (t_m \cdot l_\omega)$ is constant during all FFD executions and, hence, redundant. However, in real-world situations, this value plays a major role in determining FFD's run-time. For example: although lower values of l_ω decrease the number of FFD executions in the time between two following map-events, increasing the number of FFD instances (e.g. increasing the number of particles in a particle-filter-based SLAM implementation) causes the total run-time becoming slower. The question remains open: how do P_n , t_m , and l_ω values affect FFD's run-time in the real world? The answer to this question is presented in Section 7.

6. Maintaining and improving WFD

The WFD algorithm does not separate old and new frontiers and, therefore, has to detect them with each execution. Therefore, WFD's search domain is bounded to the whole map. In contrast, the FFD algorithm is able to detect new frontiers only. In order to enable FFD eliminating previously detected frontiers, it has to persistently perform a maintenance. The general maintenance algorithm is shown in Section 4 (Algorithm 4.5). In this section we discuss the general concepts of the maintenance process (Section 6.1) and present two algorithms which represent a hybrid version of both WFD and FFD (the former is discussed in Section 6.2 and the latter is discussed in Section 6.3).

6.1. General maintenance concepts

In Section 4.1.4, we showed and explained the maintenance stage of the FFD algorithm (Algorithm 4.5). This process can be generalized and applied to other frontier detection algorithms. By applying maintenance, we can separate between the two parts that any frontier detection needs to handle: detection of new frontiers and elimination of existing frontiers.

The presented maintenance algorithm (Algorithm 4.5) is not bounded to a specific frontier-detection algorithm. The algorithm gets its inputs and acts independently. We encourage the reader to find more details in Section 4.1.4. By integrating the maintenance algorithm into WFD, the frontier search domain can now be reduced (similar to FFD).

For instance, we integrated the maintenance mechanism into WFD algorithm and created two advanced versions of it which are presented in the following sections. The first, an incremental version of WFD, is called WFD-INC, and the second, a parallel version, which is called WFD-IP.

6.2. Incremental Wavefront Frontier Detector

Our first WFD improvement is called WFD-INC (Algorithm 6.1). In this version of WFD, instead of searching the whole map for frontier points, the search domain contains only points that lie inside the active area. The idea is based on the proof in Section 4.2; the only region in the map that contains changes (i.e. contains new frontiers and contains frontiers that are needed to be eliminated) is the active area. We define the active area to be the bounding rectangle that is constructed using the farthest laser readings received from last executions of WFD-INC.

Similar to SOTA frontier detection algorithms, WFD has to detect all frontiers in each execution. WFD has better performance than SOTA frontier-detection algorithm since it has a smaller search domain. However, comparing to FFD, it is still limited. If there were not changes in map orientation, then all previous frontiers should keep their map coordinates.

The WFD-INC algorithm exploits this fact and scans only regions that might contain changes in frontiers (i.e. regions which were hit by the laser sensors). If a map orientation was changed (lines 2–5), then all previous frontier data is cleared and the algorithm start detecting all frontiers in the same way as WFD algorithm. Another modification from original WFD algorithm is when checking the adjacent points of current scanned point (line 21). In this version of WFD, only points that lie inside the active area are scanned. This reduces the size of the search domain. The last modification from the original WFD algorithm is that a maintenance has to be called, since WFD-INC does not search over the whole map (line 28).

Both WFD (Algorithm 3.1) and WFD-INC (Algorithm 6.1) share many common code lines. We highlighted the differences between both algorithms in order to help the reader to notice the differences between them more easily.

Algorithm 6.1 Incremental Wavefront Frontier Detector (WFD-INC).**Require:** $queue_m$ // queue, used for detecting frontier points from a given map**Require:** $pose$ // current global position of the robot**Require:** $frontiersDB$ // data-structure that contains last known frontiers

```

1: function WFD-INC( $activeArea, Map, pose, frontiersDB$ )
2:   if best particle index was changed from last execution
   then
3:     clear all data from  $frontiersDB$ 
4:      $activeArea \leftarrow Map$  // active area contains all
       map points
5:   end if
6:    $queue_m \leftarrow \emptyset$ 
7:    $newFrontiers \leftarrow \emptyset$ 
8:   ENQUEUE( $queue_m, pose$ )
9:   mark  $pose$  as “Map-Open-List”

10:  while  $queue_m$  is not empty do
11:     $p \leftarrow$  DEQUEUE( $queue_m$ )
12:    if  $p$  is marked as “Map-Close-List” then
13:      continue
14:    end if
15:    if  $p$  is a frontier point then
16:       $foundFrontier \leftarrow$  EXTRACT-FRONTIER-2D( $p$ )
17:      add  $foundFrontier$  to  $newFrontiers$ 
18:      mark all points of  $foundFrontier$ 
        as “Map-Close-List”
19:    end if
20:    for all  $v \in adj(p)$  do
21:      if  $v$  not marked as {“Map-Open-List”,
        “Map-Close-List”} and  $v$  has at least one “Map-Open-
        Space” neighbor and  $v \in activeArea$  then
22:        ENQUEUE( $queue_m, v$ )
23:        mark  $v$  as “Map-Open-List”
24:      end if
25:    end for

26:    mark  $p$  as “Map-Close-List”
27:  end while
28:  MAINTAIN-FRONTIERS( $newFrontiers,$ 
     $frontiersDB, activeArea$ )
29: end function

```

6.3. Incremental-Parallel Wavefront Frontier Detector

Our second improvement of WFD is actually an improvement of WFD-INC and called WFD-IP. If changes in map orientations happen too often, then WFD-INC’s performance is the same as WFD (but now including an overhead of maintenance). Our goal is therefore to artificially reduce the number of changes in map orientation as much

Algorithm 6.2 Incremental-Parallel Wavefront Frontier Detector (WFD-IP).**Require:** $particles$ // data-structure that contains all particles of the SLAM implementation1: **for all** Particle $p \in particles$ **do**2: WFD-INC($p \rightarrow activeArea, p \rightarrow Map, p \rightarrow pose,$
 $p \rightarrow frontiersDB$)3: **end for**

as possible. The solution is simple: WFD-IP keeps a separate instance of WFD-INC algorithm for each particle. Whenever WFD-IP is executed, it executes each instance of WFD-INC according to each particle’s map data. Therefore, all instances of WFD-INC never reach lines 2–5 and keep frontier data between executions is guaranteed.

7. Experimental results

In this section, we describe the different experiments that were conducted in order to evaluate and compare the algorithms described in Sections 3, 4 and 6. In Section 7.1 we describe the settings of the testing environment. In Section 7.2 we compare all discussed frontier-detection algorithms. In Section 7.3 we examine FFD algorithm in a finer resolution and test different aspects that might affect its performance.

7.1. Background

We have fully implemented WFD, WFD-INC, WFD-IP and FFD and performed testings on data obtained from the Robotics Data Set Repository (Radish) (Howard and Roy, 2003). Figure 10 shows the environments used for the evaluation. WFD, WFD-INC, WFD-IP and FFD were compared with a SOTA (state-of-the-art) frontier detection algorithm, due to Wurm and Burgard (Wurm et al., 2008; Wurm, 2011).¹

To evaluate the algorithms, we integrated them into a single-robot exploration system. The system is based on GMapping, an open-source SLAM implementation (Grisetti et al., 2005, 2007). We integrated our code into the *ScanMatcher* component which is contained inside *gsp thread* (Grid SLAM processor). At the time that a new MapEvent is raised, all frontier-detection algorithms are executed according to current world state. Execution times are measured by Linux system call *getrusage*, which measures the CPU process time.

We used several environments taken from Radish (Howard and Roy, 2003):

- (A) Cartesium Building, University of Bremen;
- (B) Freiburg, Building 079;
- (C) Outdoor dataset recorded at the University of Freiburg;
- (D) 3rd Floor of MIT CSAIL;

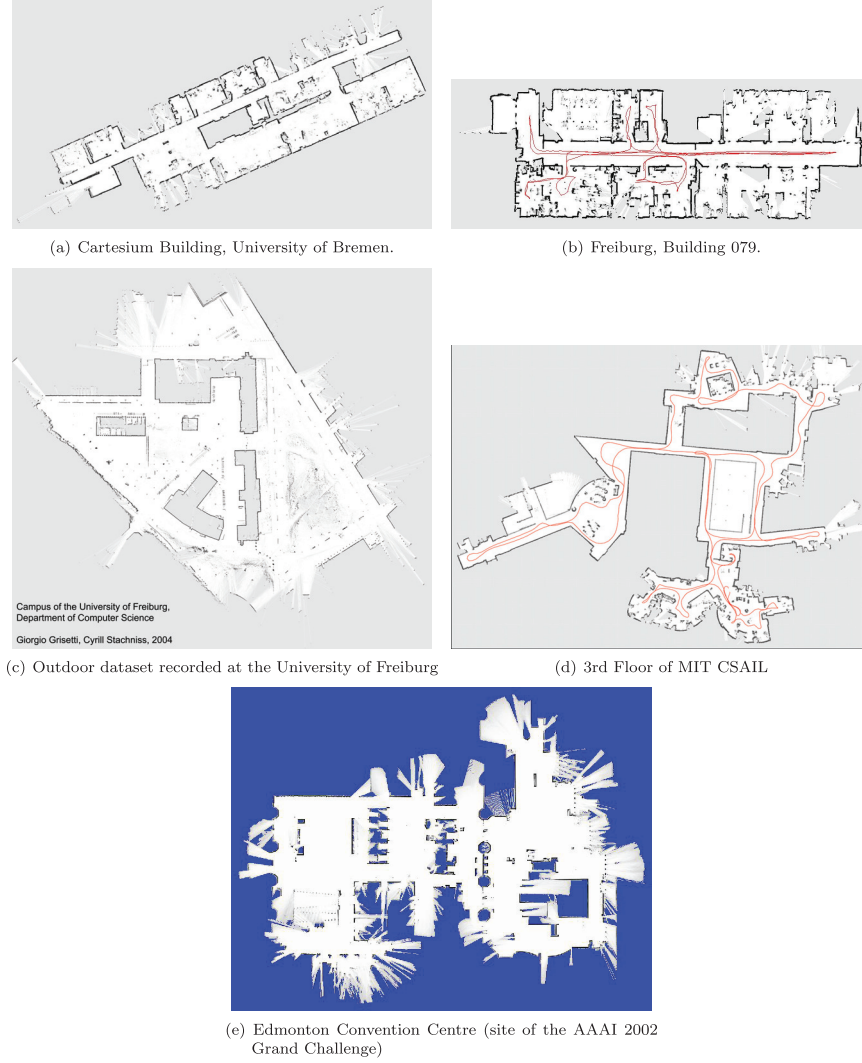


Fig. 10. Testing environments.

(E) Edmonton Convention Centre (site of the AAAI 2002 Grand Challenge).

Note that we use the exploration data (raw sensor readings and odometry) from these data sets, and thus all algorithms use exactly the same data, from the same robot trajectories. Thus, the movement of the robot is identical, and the only thing we examine is how quickly it can compute frontiers.

7.2. Comparing SOTA, FFD, WFD, WFD-INC and WFD-IP

We begin by examining overall performance. We examined the run-time of all algorithms on two different machines:

- First experiment: we used a fast desktop computer containing an Intel Core 2 Duo T6600 CPU with a clock speed of 2.20 GHz, 4 GB of RAM, L1 cache of 64 kB (8-way set-associative) and L2 cache of 2048 kB (8-way set-associative).

- Second experiment: we used a slower desktop computer containing an Intel Pentium III (Coppermine) with a clock speed of 800 MHz, 1 GB of RAM, L1 cache of 32 kB (4-way set-associative) and L2 cache of 256 kB (4-way set-associative). Research-grade robots typically have a faster CPU, but commercial robots typically do not.

FFD is called every time a new laser reading is received. Therefore, in order to compare FFD's execution time with other algorithms correctly, we accumulate FFD's execution times between calls to other algorithms. In other words, if we call WFD in time stamps t_i and t_{i+1} , then FFD's accumulated execution time is calculated by

$$\sum_{x=t_i}^{t_{i+1}} ExecutionTime_{FFD}(x)$$

Moreover, we remind the reader that because FFD is called for every particle in the particle-filtering GMapping

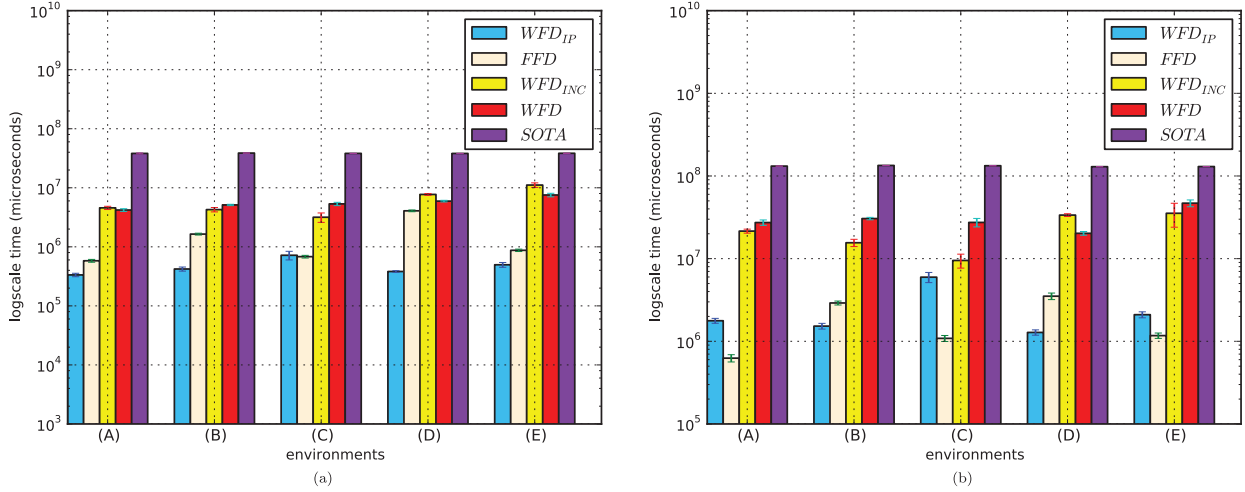


Fig. 11. Comparing all versions of WFD and FFD with the SOTA algorithm on different machines: (a) Intel T6600; (b) Intel Coppermine.

(Grisetti et al., 2007), the results here accumulate also over the number of particles (30 in our case).

Figure 11 shows results of the comparison in each of the two machines. Each group of bars represents a run over a separate map. For each algorithm, we calculate the mean execution time, over the duration of the exploration. The vertical axis measures the calculated execution time in microseconds, on a logarithmic scale. The 1-second line is at $10^6 \mu s$. The next tick, at $10^7 \mu s$, marks 10 seconds.

Figure 11 shows that WFD is faster than SOTA by approximately one order of magnitude. FFD is faster than WFD by one to two orders of magnitude. Indeed, FFD performs close to the 1-second line. In contrast, WFD and SOTA typically take anywhere from 10 to 100 seconds to perform their task, even on relatively fast machines. Surprisingly, we discovered that WFD-IP often performs frontier detection in the shortest execution time. The reason is that WFD-IP has a good balance of the frequency of number of calls (only on map-events). On the one hand, WFD algorithm has to detect all frontiers in each execution because it does not utilize previous detected frontiers. On the other hand, FFD algorithm keeps utilizing previous detected frontiers but in price of keep running the maintenance routine in the background. The WFD-IP algorithm is an hybrid approach that reduces the number of calls (relatively to FFD) while performing frontier detection on a smaller search domain (relatively to WFD).

FFD's improvement over SOTA and WFD is indeed notable, given that the measured results are not for single FFD runs, but in fact show accumulated run-time, over the frequency of the sensor readings, multiplied over the number of particles (approximately 2000 calls to FFD for each WFD or SOTA calls). These multiplicative factors have significant impact on FFD's usability. It is important to understand whether the number of particles influences the result more than the frequency of sensor readings, as the number



Fig. 12. An example of FFD's worst case as shown in environment (D).

of particles is often increased for better quality. We discuss this in detail in Section 7.3.

7.2.1. What happened in environment (D)? If we take a closer look at the results shown in Figure 11(a), in environment (D) the mean run-time of WFD is relatively not much slower than FFD. Environment (D) contains plenty of regions that seems to have small obstacles which enlarge the length of the contour scanned by FFD. In Section 5.2 we showed an upper bound of FFD's run-time complexity (Equation (26)). One of the factors that affect the boundary is the contour length that is scanned by the FFD algorithm (Section 4.1.3). Figure 12 shows an example of the enlargement of the contour when the robot arrives to a crowded region. Thus, The empirical results shown in this section well-support the theoretical boundary.

7.2.2. Why is WFD-INC sometimes worse than WFD? In the worst case, the WFD-INC algorithm should perform the

Table 1. Comparing active particle changes in different environments.

Map	Changes	Executions	Percentage (%)
(A)	56	110	50.91
(B)	52	287	18.12
(C)	25	160	15.62
(D)	193	429	44.99
(E)	73	340	21.47

same as WFD including an overhead of maintenance. However, as can be seen in Figure 11, the run-time means of WFD and WFD-INC do not have a specific trend.

We hypothesize that in certain environments, where there is a high frequency of active particle change, WFD-INC cleans its previous detected frontiers more often, and this explains its run-time.

We provide evidence to support this hypothesis in Table 1 and Figure 13. Table 1 shows the particle changes in different environments. Each row represents a single environment. The first column contains the environment identifier. The second column contains the number of active particle changes. The third column contains the total number of executions and the forth column contains the percentage of particle changes relative to the total number of executions.

Figure 13 shows the active particle changes over time. For a specific environment, the vertical axis represents each execution and the horizontal axis represents whether the active particle was changed (*Yes* for a change and *No* otherwise). There is an exception in environment (C) in which the active particle changes percent is quite low but WFD-INC's execution time is still worse than WFD's execution time. Here, the explored environment is very large and, hence, for each change, the robot has to detect frontiers in a very large area, in contrast to other environments.

7.2.3. What happened to WFD-IP in environments (A), (C) and (E), in Figure 11(b)? Figure 11 shows the run-time results of all algorithms on two testing machines. On the stronger machine (Figure 11(a)) WFD-IP outperforms all other algorithms. However, on the weaker machine (Figure 11(b)), the situation is different in environments (A), (C) and (E).

The two machines differ in both memory size and the number of cores. The Coppermine machine is equipped with less random access memory (1 GB) than the stronger machine (4 GB). The key feature of WFD-IP is its robustness against changes in active particle since it holds a separate instance of WFD-INC for each particle. In our implementation, there are 30 particles and hence, WFD-IP holds 30 instances of WFD-INC (i.e. 30 frontier databases, 30 maps, etc.). In addition, the Coppermine machine is equipped with a single core CPU, in contrast to the stronger machine that is equipped with two cores. The GMapping SLAM system runs two threads (a *main* thread and a SLAM

thread). We believe that the stronger machine allows each thread to run on a separate core.

There thus remains a question as to the reason for such behavior of WFD-IP relates to either the lack of memory or to the single core of the weaker machine. To determine this, we bounded the memory of the strong machine to be 1 GB. The behavior of WFD-IP remained the same (we forgo adding the redundant figure here). The conclusion is therefore that the number of cores caused the differences in WFD-IP's behavior. This should be taken into account when choosing between FFD and WFD-IP for a given robot CPU.

7.3. Evaluating FFD at a finer resolution

We turn to evaluating FFD more closely. Figure 14 compares the run-time of individual particles in specific environments. Each bar represents a specific particle. The vertical axis measures the mean run-time of FFD for the particle. The error-bars represent the standard deviation of each particle's run-time.

The figure shows that the per-particle run-time is measured in a few hundred microseconds. Thus, the overall results were accumulating comparing the accumulation of thousands of FFD runs against single WFD and SOTA runs. Indeed, one can boost FFD's execution time by not executing it on every received laser reading, since the frequency of receiving new laser readings is often higher than the speed of processing and updating the map anyways. Many laser sensors generate output at 30–75 Hz, at least three times faster than the rate at which the robots process the information. By ignoring some laser readings, FFD would perform much better, without any noticeable decay in mapping quality.

FFD's performance according to number of particles

One of FFD's drawbacks is that in order to get a complete picture of frontiers in a given time, it has to persistently run in the background. As mentioned in Section 4.1.4, if FFD is integrated into a particle-filter-based SLAM implementation, each particle has its own instance of the FFD algorithm and, hence, the overall run-time is increased. Figure 15 compares the mean run-time of FFD by changing the number of particles in specific environments. Each bar represents a run of FFD with a specific number of particles. The vertical axis measures the mean run-time of FFD for the configuration. The error-bars represent the standard deviation of each configuration's run-time.

Speeding up WFD even further WFD's execution time can be boosted even more by reducing the grid size (i.e. by using a coarse grid). Of course, there is a trade-off between shorter execution time and the quality of the output frontiers, even though standard exploration tasks can utilize the output frontiers received in this manner. The grid is divided into blocks in size of the robot's width and height. Smaller blocks will not make sure that robot will be able to pass

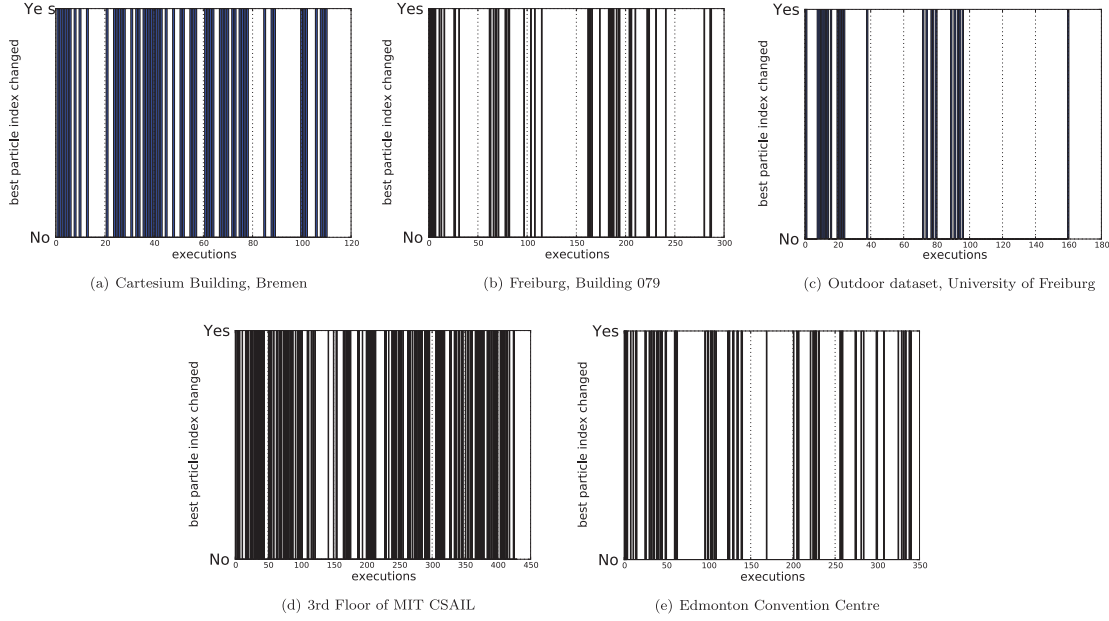


Fig. 13. Changes in active particle over executions.

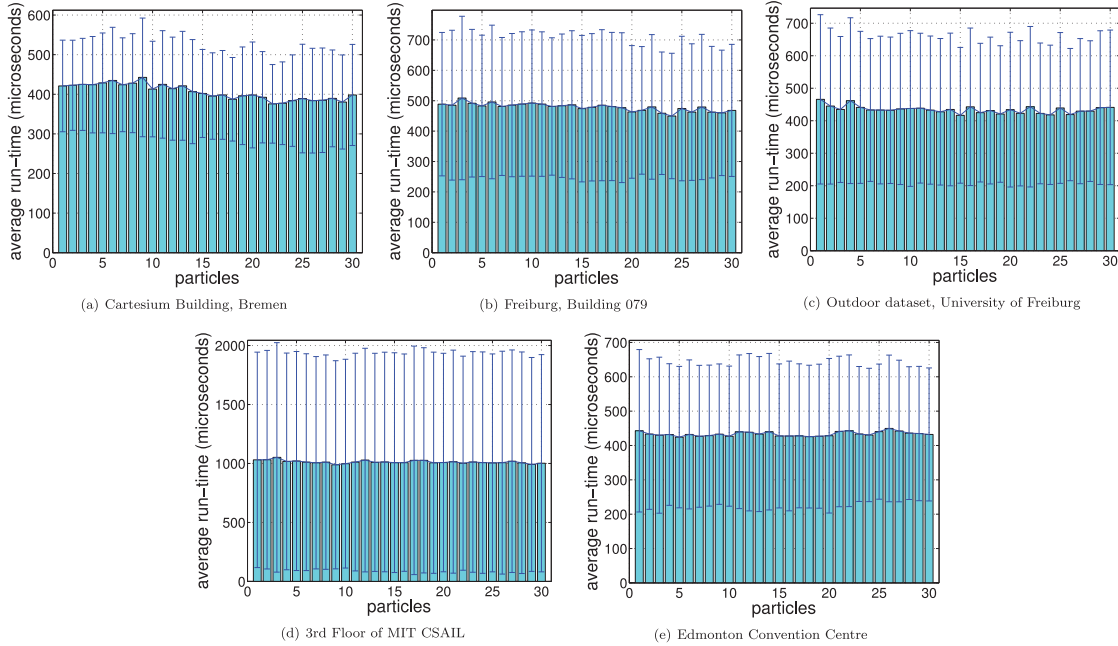


Fig. 14. FFD run-time for individual SLAM particles.

through terrain obstacles (i.e. corridors). Each block in the real world is represented by a single cell in the reduced grid. In order to determine the occupancy value of the cell, we examined different strategies. We considered both the speed of creating the new grid and the quality of the output. We found out that sampling the center of the block edges and the block center yields the best results. We plan to investigate efficient techniques to reduce the grid size while considering the quality of the output coarse grid data.

8. Conclusions and future work

Frontier-based exploration is the most common approach to solve the exploration problem. SOTA frontier detection methods process the entire map data which hangs the exploration system for a few seconds with every call to the frontier-detection algorithm.

In this work we present four novel faster frontier detectors, WFD, FFD, WFD-INC and WFD-IP. The first algorithm, a graph-based search, processes the map points

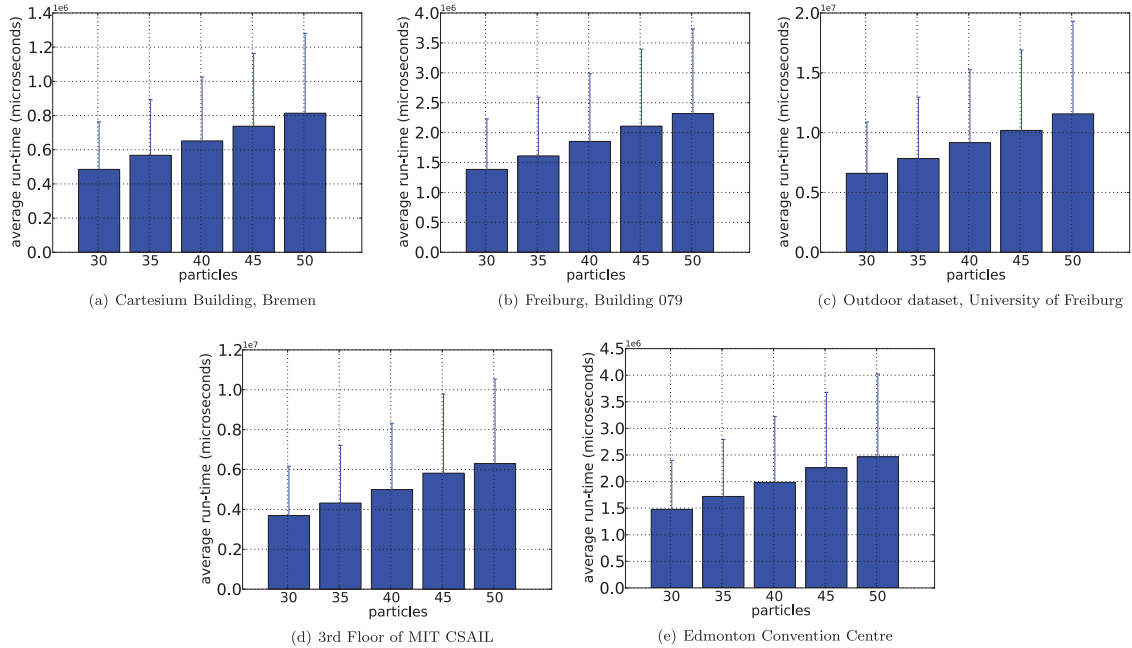


Fig. 15. FFD's run-time according to the number of particles.

which have already been scanned by the robot sensors and, therefore, does not process unknown regions in each run (although it grows more slowly as more area is known). The second algorithm, a laser-based approach for frontier detection, only processes new laser readings which are received in real time, eliminating also much of the known search area. However, maintaining previous frontiers knowledge requires tight integration with the mapping component, which may not be straightforward. The third and fourth algorithms are a combined approach of both WFD and FFD. Both algorithms search for frontiers within the known regions but their search space is still smaller than WFD's search space since they search for frontiers only in the regions that were covered by the robot sensors since their last execution.

We have described efficient implementation for all algorithms and compared them empirically. FFD and WFD-IP are shown to outperform WFD, WFD-INC by one to two orders of magnitude. In addition, FFD and WFD-IP outperform SOTA by two to three orders of magnitude.

In the future, we plan to integrate the general maintenance mechanism with EKF-based SLAM implementations, which we hope will lead to further improvements. We also plan to begin investigation of novel exploration policies, based on real-time frontier detection.

Note

1. We thank Kai M. Wurm and Wolfram Burgard for providing us with their own implementation of a SOTA frontier-detection algorithm.

Funding

This research was supported in part by IMOD and ISF grant number 1511/12.

References

- Apostolopoulos D, Pedersen L, Shamah B, Shillcutt K, Wagner M and Whittaker W (2001) Robotic antarctic meteorite search: Outcomes. In: *Proceedings of IEEE International conference on robotics and automation (ICRA-01)*, pp. 4174–4179.
- Batalin M and Sukhatme G (2003) Efficient exploration without localization. In: *Proceedings of IEEE International conference on robotics and automation (ICRA-03)*, volume 2. IEEE, pp. 2714–2719.
- Berhault M, Huang H, Keskinocak P, et al. (2003) Robot exploration with combinatorial auctions. In: *Proceedings of the IEEE/RSJ International conference on intelligent robots and systems (IROS-03)*, pp. 1957–1962.
- Bouraqadi N, Doniec A and de Douai EM (2009) Flocking-based multi-robot exploration. In: *National conference on control architectures of robots*.
- Bresenham J (2010) Algorithm for computer control of a digital plotter. *IBM Systems Journal* 4(1): 25–30.
- Burgard W, Moors M, Fox D, Simmons R and Thrun S (2000) Collaborative multi-robot exploration. In: *IEEE International conference on robotics and automation*, Vol. 1, pp. 476–481.
- Burgard W, Moors M, Stachniss C and Schneider F (2005) Coordinated multi-robot exploration. *IEEE Transactions on Robotics* 21(3): 376–378.
- Calisi D, Farinelli A, Iocchi L and Nardi D (2007) Multi-objective exploration and search for autonomous rescue robots. *Journal of Field Robotics* 24(8-9): 763–777.
- Cormen TH, Leiserson CE, Rivest RL and Stein C (2001) *Introduction to Algorithms*. Cambridge, MA: MIT Press.

- Durrant-Whyte H and Bailey T (2006) Simultaneous localisation and mapping (SLAM): Part I the essential algorithms. *Robotics and Automation Magazine* 13(99): 80.
- Fox D, Ko J, Konolige K, Limketka B, Schulz D and Stewart B (2010) Distributed multi-robot exploration and mapping. *Proceedings of the IEEE* 94(1): 1325–1339.
- Geyer C (1992) Practical Markov chain Monte Carlo. *Statistical Science* 7(4): 473–483.
- Grisetti G, Stachniss C and Burgard W (2005) Improving grid-based SLAM with Rao-Blackwellized particle filters by adaptive proposals and selective resampling. In: *Proceedings of IEEE International conference on robotics and automation (ICRA-05)*, pp. 2443–2448.
- Grisetti G, Stachniss C and Burgard W (2007) Improved techniques for grid mapping with Rao-Blackwellized particle filters. *IEEE Transactions on Robotics* 23: 34–46.
- Hastings WK (1970) Monte Carlo sampling methods using Markov chains and their applications. *Biometrika* 57(1): 97–109. DOI: 10.1093/biomet/57.1.97.
- Hougen DF, Benjaafar S, Bonney J, et al. (2000) A miniature robotic system for reconnaissance and surveillance. In: *Proceedings of IEEE International Conference on robotics and automation (ICRA-00)*, pp. 501–507.
- Howard A and Roy N (2003) The robotics data set repository (RADISH). <http://radish.sourceforge.net/>.
- Juliá M, Gil A and Reinoso O (2012) A comparison of path planning strategies for autonomous exploration and mapping of unknown environments. *Autonomous Robots* 33(4): 427–444.
- Kitano H, Tadokoro S, Noda I, et al. (1999) Robocup rescue: Search and rescue in large-scale disasters as a domain for autonomous agents research. In: *Proceedings of IEEE International conference on systems, man and cybernetics (SMC-99)*. IEEE Computer Society, pp. 739–746.
- Ko J, Stewart B, Fox D, Konolige K and Limketkai B (2003) A practical, decision-theoretic approach to multi-robot mapping and exploration. In: *Proceedings of the IEEE/RSJ International conference on intelligent robots and systems (IROS-03)*, pp. 3232–3238.
- Konolige K, Fox D, Ortiz C, et al. (2006) Centibots: Very large scale distributed robotic teams. In: *Experimental Robotics IX*, pp. 131–140.
- Konolige K, Ortiz C, Vincent R, et al. (2003) Large scale robot teams. In: *Proceedings from the 2003 International workshop on multi-robot systems: from swarms to intelligent autonomy*, vol. 2. Berlin: Springer, pp. 193–204.
- Kuipers B and Byun Y (1991) A robot exploration and mapping strategy based on a semantic hierarchy of spatial representations. *Robotics and Autonomous Systems* 8(1–2): 47–63.
- Lau H (2003) Behavioural approach for multi-robot exploration. In: *Australasian conference on robotics and automation (ACRA)*, Brisbane.
- Leonard J and Durrant-Whyte H (1991) Mobile robot localization by tracking geometric beacons. *IEEE Transactions on Robotics and Automation* 7(3): 376–382.
- Neal RM (1993) *Probabilistic Inference Using Markov Chain Monte Carlo Methods*. Technical Report CRG-TR-93-1, Department of Computer Science, University of Toronto.
- Puig Valls D (2010) Balanced multi-robot exploration through a global optimization strategy. *Journal of Physical Agents* 4(1): 35–44.
- Rekleitis I, Dudek G and Milios E (1997) Multi-robot exploration of an unknown environment, efficiently reducing the odometry error. In: *International joint conference on artificial intelligence (IJCAI)*, vol. 15, pp. 1340–1345.
- Rekleitis I, Dudek G and Milios E (2001) Multi-robot collaboration for robust exploration. *Annals of Mathematics and Artificial Intelligence* 31(1): 7–40.
- Sawhney R, Krishna KM and Srinathan K (2009) On fast exploration in 2D and 3D terrains with multiple robots. In: *Proceedings of the eighth International joint conference on autonomous agents and multi-agent systems (AAMAS-09)*, pp. 73–80.
- Simmons R, Apfelbaum D, Burgard W, et al. (2000) Coordination for multi-robot exploration and mapping. In: *Proceedings of the seventeenth National conference on artificial intelligence (AAAI-00)*, pp. 852–858.
- Stachniss C (2006) *Exploration and Mapping with Mobile Robots*. Ph.D. thesis, University of Freiburg, Department of Computer Science.
- Stachniss C, Mozos O and Burgard W (2006) Speeding-up multi-robot exploration by considering semantic place information. In: *Proceedings of IEEE International conference on robotics and automation (ICRA-06)*. IEEE, pp. 1692–1697.
- Stoeter S, Rybski P, Erickson M, et al. (2000) A robot team for exploration and surveillance: design and architecture. In: *Proceedings of the International conference on intelligent autonomous systems*, pp. 767–774.
- Thrun S, Burgard W and Fox D (2005) *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents Series)*. Cambridge, MA: The MIT Press.
- Visser A (2011) Personal communication by email, 4 January.
- Visser A and Slamet BA (2008) Including communication success in the estimation of information gain for multi-robot exploration. In: *Proceedings of the 6th International symposium on modeling and optimization in mobile, ad hoc, and wireless networks (WiOpt-08)*, pp. 680–687. DOI: 10.1109/wiopt.2008.4586160.
- Welch G and Bishop G (1995) *An Introduction to the Kalman Filter*. Technical Report, University of North Carolina at Chapel Hill, Chapel Hill, NC. http://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf.
- Wurm K (2011) Personal communication by email, 20 January.
- Wurm K, Stachniss C and Burgard W (2008) Coordinated multi-robot exploration using a segmentation of the environment. In: *Proceedings of the IEEE/RSJ International conference on intelligent robots and systems (IROS-08)*, Nice, France.
- Yamauchi B (1997) A frontier-based approach for autonomous exploration. In: *Proceedings of the 1997 IEEE International symposium on computational intelligence in robotics and automation (CIRA-97)*. Washington, DC: IEEE Computer Society, pp. 146–151.
- Yamauchi B (1998) Frontier-based exploration using multiple robots. In: *Proceedings of the second International conference on autonomous agents (Agents-98)*, pp. 47–53.
- Zlot R, Stentz A, Dias M and Thayer S (2002) Multi-robot exploration controlled by a market economy. In: *Proceedings of IEEE International conference on robotics and automation (ICRA-02)*, volume 3. IEEE, pp. 3016–3023.